Klinikum rechts der Isar
Technische Universität München

Multi-Modal Deep Learning

Lecture Series | Summer 2025

Institute for AI and Informatics in Medicine

13.05.2025 and 20.05.2025

WHAT WE HAVE COVERED SO FAR

- **Manifold Hypothesis**: Why does (multimodal) deep learning work
- **Multimodal Deep Learning Pipelines**: Overview of multimodal approaches
- **Types of Data Structures**: How do different modalities differ
- **Representations/Features/Encodings**: Preparing modalities for deep learning

What will be next?
(Unimodal) Encoders/Decoders for different modalities

Why does it matter?

- We need to understand how to encode individual modalities before combining them
- We need to understand the properties of different encoders to interconnect them
- Multimodal encoders/decoders use similar components as unimodal encoders

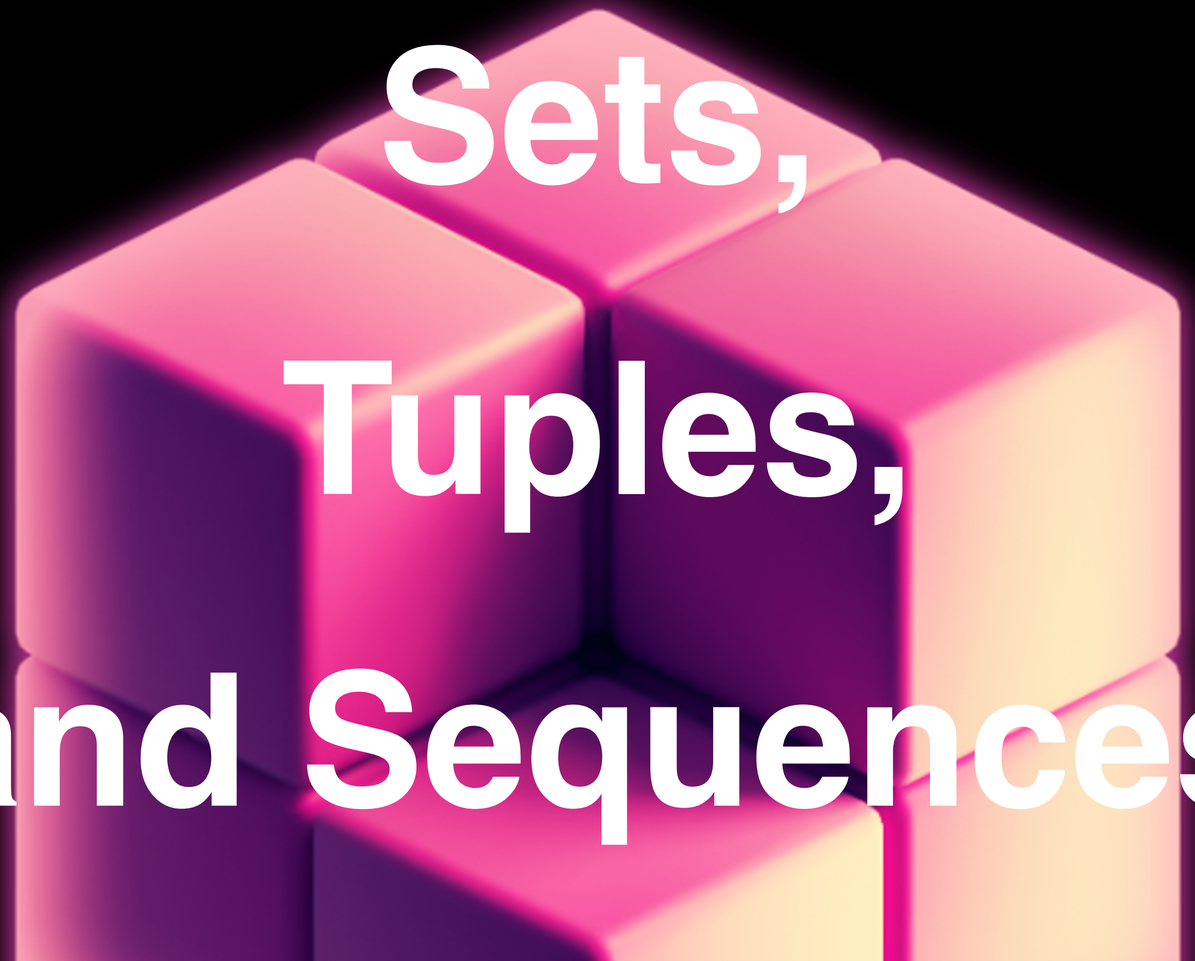
WHAT WILL BE COVERED TODAY

Encoders/Decoders for Sets, Tuples, and Sequences

- Sets/Tuples/Sequences are widespread input/output modalities
- Sets/Tuples/Sequences are commonly used as structures within multimodal models
  - Intermediate representations are often sets/tuples/sequences
- Components from Set/Tuple/Sequence encoders are commonly used in multimodal models
  - Enable interaction/conversion between modalities

➡ **Will be an important building block for multimodal pipelines**
(last two weeks of the semester)

DL on Sets, Tuples, and Sequences

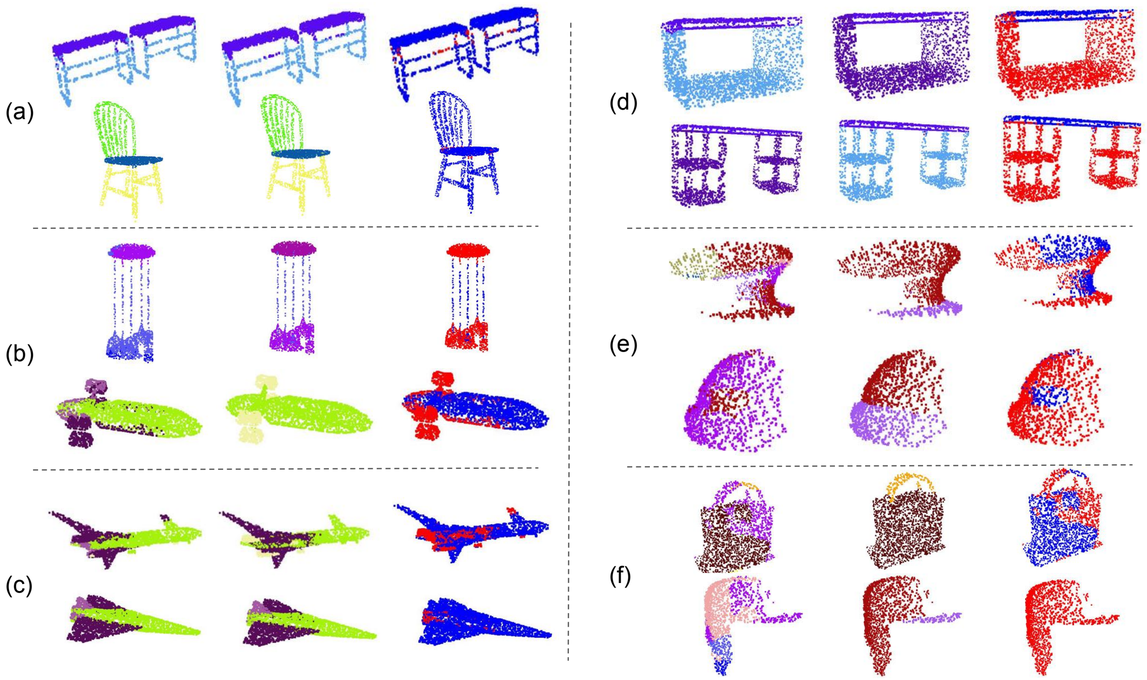


EXAMPLES OF SETS, TUPLES, AND SEQUENCES IN DEEP LEARNING

SETS AS INPUTS

Point Clouds

- Set of points (each with coordinates and optional additional features)
- Example: LiDAR: (x, y, z)



SETS AS INPUTS

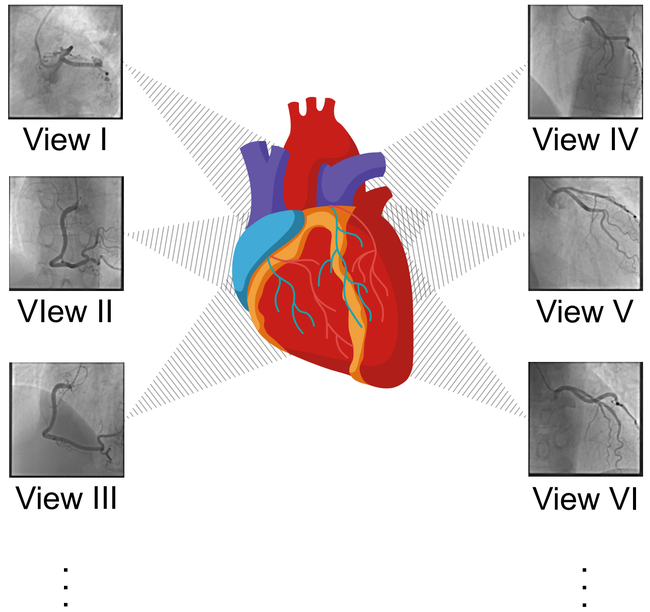
Object Clusters

- Set of objects belonging to the same cluster
- Example: **Text concept retrieval**
  - Given a set of words: **{tiger, lion, cheetah}**
  - Predict related words: e.g. **jaguar, puma**

SETS AS INPUTS

Multiple views

- Set of views of the same scene
- Example: **Multiview diagnosis**
  - Given multiple radiology images (e.g. X-rays) with different views all from the same study
  - Diagnose the study



SETS AS OUTPUTS

Tagging / Multilabel Classification

- Assign a set of tags to an input (e.g. an image)
- Unlike image multiclass classification, tags are non-exclusive => multilabel binary

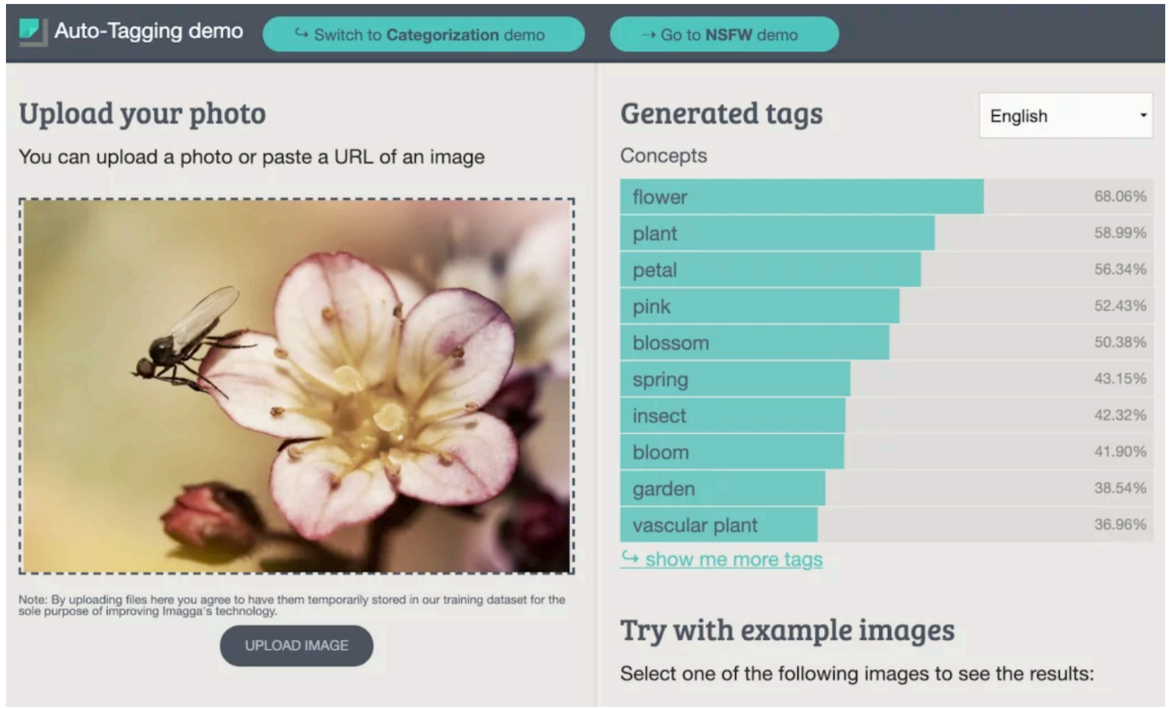


Image source: https://imagga.com/blog/image-tagging/

SETS AS OUTPUTS

Object Detection

- Identify and localize the set of objects present in an image
- => Predict set of bounding boxes (with associated classes)

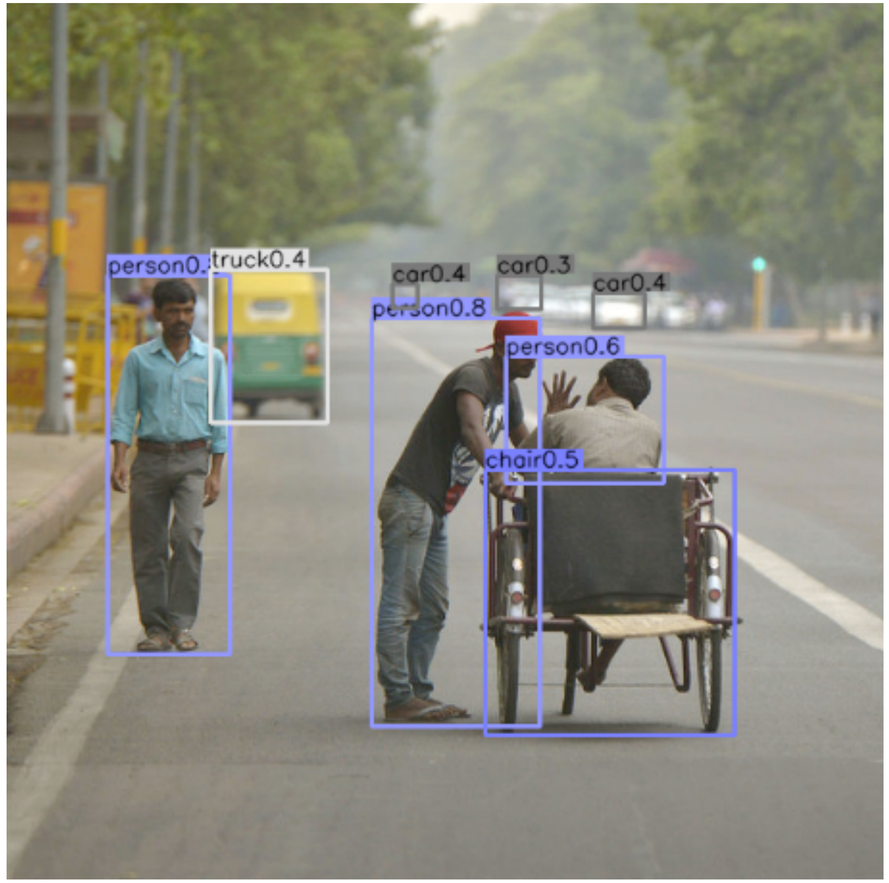


Image source: Huggingface

SETS AS OUTPUTS

Instance Segmentation

- Identify and localize the set of objects present in an image
- => Predict set of masks (with associated classes)
- Like object detection but with masks

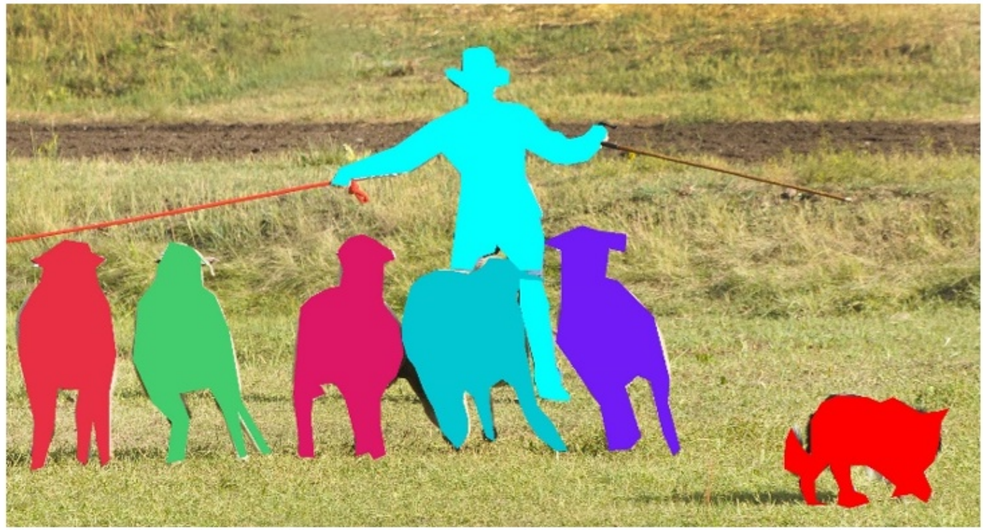


Image source: COCO Dataset (https://arxiv.org/abs/1405.0312v3)

TUPLES AS INPUTS

Tabular Data (Tuple of columns)

Smoker	Age	BP	BMI	Sex	Fitness	Alcohol
FALSE	62	150/90	29.2	Male	High	Moderate

TUPLES AS OUTPUTS

Multitask Classification / Regression

- Predicting several aspects of a single input
- Like predicting a tabular row
  - Classification tasks => discrete column
  - Regression tasks => numeric column

Smoker	Age	BP	BMI	Sex	Fitness	Alcohol
FALSE	62	150/90	29.2	Male	High	Moderate

SEQUENCES AS INPUTS (OR OUTPUTS)

Text (Sequence of words)

- Books / documents
- Chats
- Medical Texts (e.g. discharge summaries)

Discharge Summary

Admission Date: 08/01/2020
Discharge Date: 08/04/2020
Date of Birth: 04/05/1962
Sex: Female
History of Present Illness: Malesuada proin libero nunc consequat interdum varius sit amet mattis. Justo nec ultrices dui sapien eget mi proin sed libero. Vitae nunc sed velit dignissim sodales ut eu. Iaculis nunc sed augue lacus viverra vitae.
Past Medical History: Viverra maecenas accumsan lacus vel. A diam sollicitudin tempor id eu nisl nunc. Egestas integer eget aliquet nibh praesent tristique magna sit amet.
Principal Diagnosis: Pneumonia
Brief Hospital Course: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Sit amet luctus venenatis lectus magna fringilla. Nunc lobortis mattis aliquam faucibus. Ut tortor pretium viverra suspendisse potenti nullam. Sociis natoque penatibus et magnis dis. Nunc pulvinar sapien et ligula ullamcorper malesuada proin libero. Maecenas sed enim ut sem viverra aliquet. Mauris in aliquam sem fringilla ut. Habitasse platea dictumst vestibulum rhoncus est pellentesque elit ullamcorper dignissim. Consectetur a erat nam at lectus. Eget nullam non nisi est sit amet facilisis magna. Nulla porttitor massa id neque aliquam. Ultrices dui sapien eget mi proin. Quis commodo odio aenean sed adipiscing diam donec.
Medications at Admission: Benadryl, Trazadone 50mg, Aspirin 20mg, Vitamin E 500mg
Discharge Medications: Zolpidem 5mg, Trazadone 50mg, Acetaminophen 500mg, Azithromycin 10mg
Follow-up Plans: Return to your PCP in 2 weeks. You will need a cat scan.

SEQUENCES AS INPUTS (OR OUTPUTS)

RNA / DNA / Amino-acid sequences

```
>Fu9D-ITS1F_D20.ab1
NNNNNNNNNANCTGCGGAAGGNNCATTATTGAATAAACATGGTTGGGTTGTAGCTGACTCCTTGGAGTATGTGCACACCT
GCCGTCTTTATCTATCCACCTGTGCACACATTGTAGTCTTGGGGGATTGATTAGTGACAAATTTTGTTGCAATGTCGTCC
TCCGAGGTCTATGTTATCATAAACCACTTAGTATGTCGTAGAATGAAGTATTTGGGCCTTAGTGCCTATAAAACAAAATA
CAACTTTCAGCAACGGATCTCTTGGCTCTCGCATCGATGAAGAACGCAGCGAAATGCGATAAGTAATGTGAATTGCAGAA
TTCAGTGAATCATCGAATCTTTGAACGCACCTTGCGCTCCTTGGTATTCCGAGGAGCATGCCTGTTTGAGTGTCATTAAA
TTCTCAACCCCTCCAGCTTTTGTGGCTGGTCGTGGCTTGGATATGGGAGTTTTGCTGGTCTCATTTCGAGATCAGCTCTC
CTGAAATGCATTAGTGGAACCGTTTGCGATCCGTCACCGGTGTGATAATTATCTACGCCATAGACTGTGAACGCTCTCTG
TATTGTTCTGCTTCTAACTGTCCTGTAAAAGGACAACGATATTGAACTTTTGACCTCAAATCAGGTAGGACTACCCGCTG
AACTTAAGCATANNNNNN
```

SEQUENCES AS INPUTS

Longitudinal Data (Sequence of Time Points)

- E.g. Electronic health records (EHR)
- Each visit is one element of the sequence

Image source: https://www.nature.com/articles/s41598-020-62922-y

SEQUENCES AS OUTPUTS

Robotics: Action or Path/Trajectory Planning

- Predict sequence of
  - actions (discrete elements)
  - or positions (continuous elements)

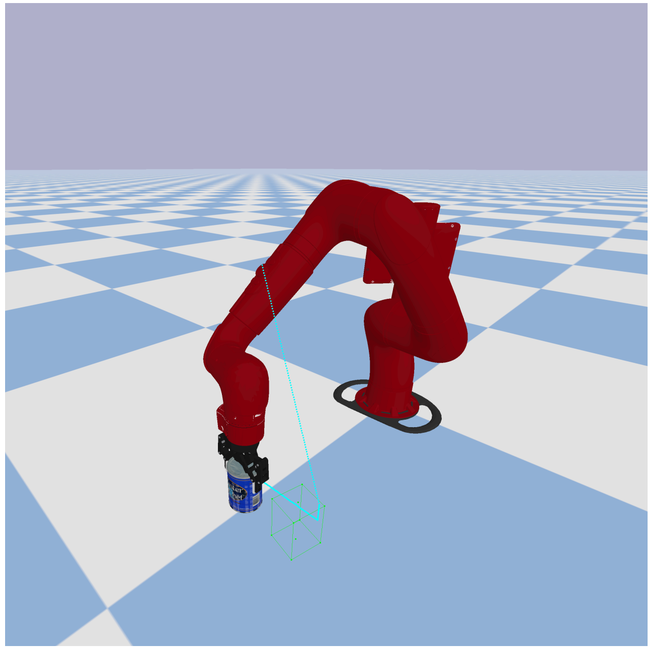


Image source: https://github.com/kwonathan/language-models-trajectory-generators?tab=readme-ov-file

SIGNALS / TIME SERIES AS INPUTS

Signals = continuous sequences

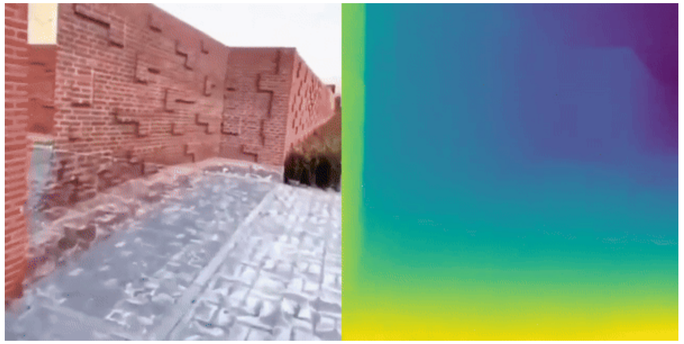
Examples

- Voice (different languages)
- Environmental Sounds
- Music
- ECG (Medical)
- EEG (Medical)
- Sensory data or other time series (electronics, mechanical, finance, geology, …)

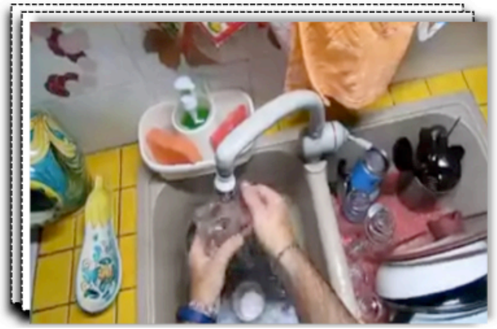
SIGNALS / TIME SERIES AS INPUTS

Videos

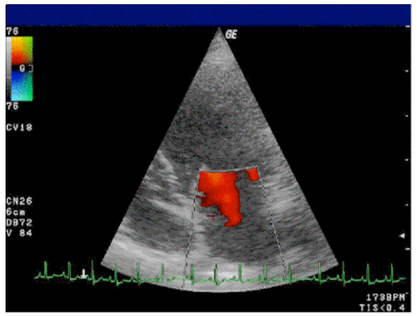
“Natural Videos”
(Web Videos)



Depth Videos



Egocentric Videos



2D Ultrasound
(Medical)

AGENDA

- Properties of Sets, Sequences, and Tuples
- Encoding Sets, Sequences, and Tuples
- Representing anything as a Set
- Decoding/Predicting Sets, Sequences, and Tuples
- Real-world Architectures
- Tabular Data

KEY PROPERTIES OF SETS, TUPLES, AND SEQUENCES

SETS

Sets in mathematics:

- Discrete collection of elements
- Cardinality can be finite, countable or uncountable infinite
- Elements can be anything

SETS

Sets in mathematics:

- Cardinality can be finite, countable or uncountable infinite

Set inputs / outputs in deep learning practice:

$$\{a_1, \ldots, a_n\}_{n \in \mathbb{N}, n \leq N_{\max}}$$

- Cardinality must be finite ($n \in \mathbb{N}$) and is typically bound ($n \leq N_{\max}$)
  - cardinality may or may not be bound by the architecture
  - cardinality will always be bound by physical constraints (memory, …)
  - cardinality also bound by available training data
  - **BUT: cardinality can differ between samples**

SETS

Sets in mathematics:

- Elements can be anything

Set inputs / outputs in deep learning practice:

$$\{a_1, \ldots, a_n\}_{a_i \in \mathcal{T}}$$

- Elements all have the same type $\mathcal{T}$ ($a_i \in \mathcal{T}$)
  - we need to be able to encode/embed the individual elements
  - **BUT: we can also support multiple types as union types** $\mathcal{T} = \mathcal{T}_1 \cup \mathcal{T}_2$
  - we use different encoders/embeddings for each type
  - we may optionally tag the type of each element $\{(a_1, t_1), \ldots, (a_n, t_n)\}$

MULTIPLICITY: FROM SETS TO MULTISETS

Sets: Elements are unique, i.e. multiplicity of elements does not matter

$$\{a, b, c\} = \{a, a, b, c\}$$

Multisets: Elements may occur with different multiplicities $m(a_i)$

$$\{a^1, b^1, c^1\} \neq \{a^2, b^1, c^1\}$$

In practice, we often do not distinguish between sets and multisets and do not force uniqueness.

ORDERING: FROM MULTISETS TO SEQUENCES

Sets/Multisets: Elements are not ordered, i.e. the ordering does not matter

$$\{a, b, c\} = \{c, b, a\}$$

$$\left\{a^2, b^1, c^1\right\} = \left\{c^1, a^2, b^1\right\} = \{a, b, c, a\}$$

Sequences: Elements are ordered

$$\left(a, b, c\right) \neq \left(c, b, a\right)$$

SEQUENCES

Sequences in deep learning practice: $\left(a_i\right)_{i=1}^{n}$

- Length must be finite ($n \in \mathbb{N}$) and is typically bound ($n \leq N_{\max}$)
  - length is typically not explicitly bound by the architecture
  - length will always be bound by physical constraints (memory, …)
  - length also bound by available training data
  - **BUT: length can differ between samples**

- Elements all have the same type $\mathcal{T}$ ($a_i \in \mathcal{T}$)
  - as for sets

SIZES: FROM SEQUENCES TO TUPLES

Sequences: Lengths of different samples may differ

(a, b, c) and (a, b, c, d) are different sequences

but may both be inputs/outputs of the same model

Tuples: All samples have the exact same length (as defined by the model)

(a, b, c) would be an input of a triplet (3-tuple) encoder

(a, b, c, d) would be an input of a quadruple (4-tuple) encoder

=> Sequences are variable-length tuples / tuples are fixed-length sequences

TUPLES

Tuples in deep learning practice: $(a_1, \ldots, a_k)$

- Length is fixed and finite by design ($k \in \mathbb{N}$)
  - **length is constant for different samples => no max-length required**
- Elements can have different types: ($a_i \in \mathcal{T}_i$)
  - no union type (e.g. $\mathcal{T} = \mathcal{T}_1 \cup \mathcal{T}_2$) required as for sets or sequences
  - different embeddings/encodings per element possible
  - **BUT: the type $\mathcal{T}_i$ at each index i is always the same for all samples**

TUPLES: RECORDS AND TABULAR DATA

Records / Tabular Data are Tuples!

- Elements are identified by names (columns) instead of indices
- Names may be included in the values a_i (consider their semantic meaning) or can be treated as indices (ignoring their semantic meaning)

Different columns may have different feature dimensions

- continuous columns
- discrete columns

SIGNALS: FROM DISCRETE TO CONTINUOUS INDICES

Sequences: Indices are integers (i.e. discrete)

- Within a range of indices, there is a finite number of elements

Signals / time series: Indices are real-valued (i.e. continuous)

- Within a range of indices, there is an infinite number of elements
- cannot be fully stored or encoded

➡ **Signals need to be discretely sampled for processing and encoding**

➡ **Discretely sampled signals can (mostly) be treated like sequences!**

Multivariate signals: Some signals have multiple variates / channels

- Multiple real values per time point

➡ **Encoding schema required to handle multi-variate data**

SETS, MULTISETS, SEQUENCES, TUPLES, SIGNALS

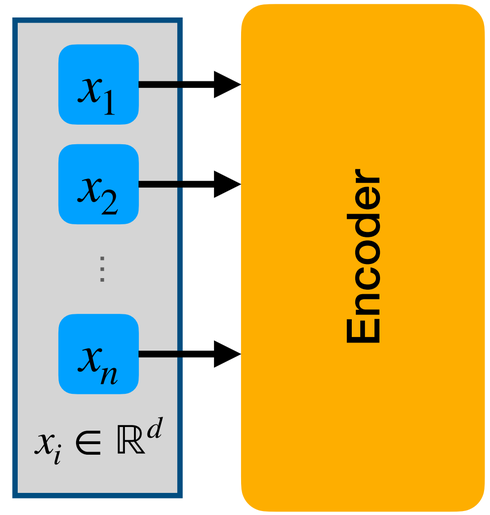
	Set	Multiset	Sequence	Tuple	Signals
Notation	$\{x_1, \ldots, x_n\}$	$\left\{x_1^{m(x_1)}, \ldots, x_n^{m(x_n)}\right\}$	$(x_i)_{i=1}^{n}$	$(x_1, \ldots, x_k)$	$f(t)$
Examples	$\{a, b, c\}$	$\{a^2, b^1, c^3\}$ $= \{a, a, b, c, c, c\}$	$(a, b, a, b, \ldots)$	(a, c, b, a)	
Multplicity	no	yes	yes	yes	yes
Ordered	no	no	yes	yes	yes
Length	dynamic	dynamic	dynamic	fixed, finite	dynamic
Element Indices	discrete $n \in \mathbb{N}$	discrete $n \in \mathbb{N}$	discrete $n \in \mathbb{N}$	discrete $k \in \mathbb{N}$	continuous $t \in \mathbb{R}$

Encoding Inputs

HOW TO ENCODE MULTI-VALUED INPUTS?

Multi-valued Model Inputs

- Single input **x** consists of multiple individual **elements** x_i
- Each element is represented as a feature vector $x_i \in \mathbb{R}^d$
  - Assume d is constant for all i (for now)

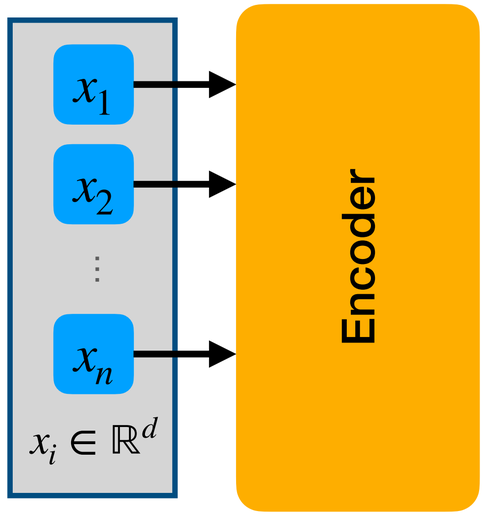


Multi-valued Input (Set/Sequence/Tuple)

HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

How are different x_i related?

- What is the structure of input **x**?
  - Set / Sequence / Tuple
  - Length



Multi-valued Input (Set/Sequence/Tuple)

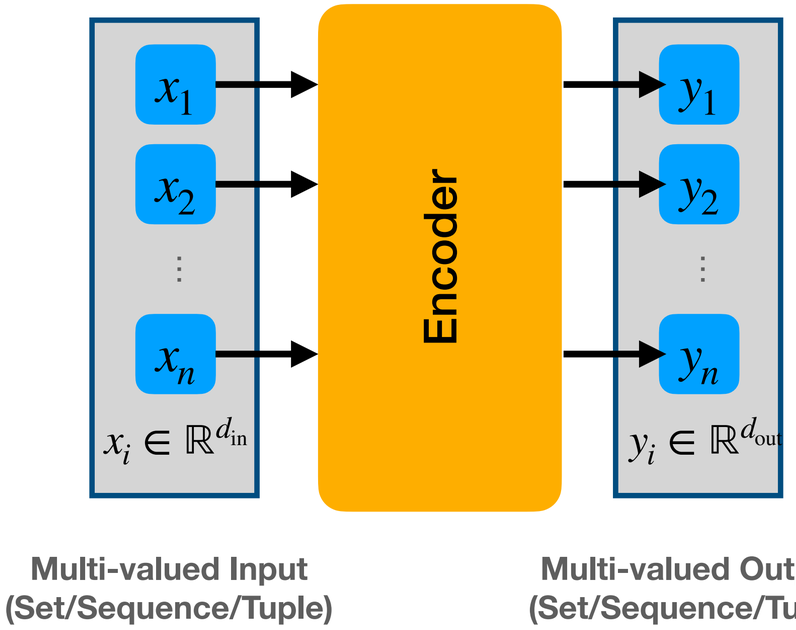
HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

How are different x_i related ?

- What is the structure of input **x**?
  - Set / Sequence / Tuple
  - Length

Do we keep the same structure in the output y?

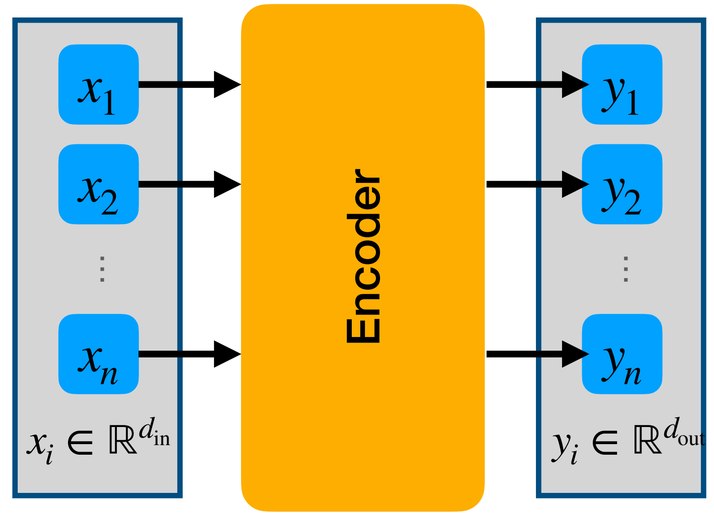
- Structure of **y** is the same as of **x**
  - E.g. if **x** represents set of n elements then so does **y**
- There is a meaningful path from each x_i to each y_i
  - if x_i contains features of an element i, then y_i contains (more high-level) features of the same element i



HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

Permutation-Equivariance

- If input order changes, the output order changes
  - Output features are not affected, only reordered
  - Encoder must ignore order when processing features!
- Given permutation π
  - Permuted input $\mathbf{x}' = \pi(\mathbf{x})$
  - Processed original input: $\mathbf{y} = \text{Encoder}(\mathbf{x})$
  - Processed permuted input: $\mathbf{y}' = \text{Encoder}(\mathbf{x}')$
  - Equivariance: $\mathbf{y}' = \pi(\mathbf{y})$



$$\text{Encoder}(\pi(\mathbf{x})) = \pi\left(\text{Encoder}(\mathbf{x})\right)$$

or $$\text{Encoder}(\mathbf{x}) = \pi^{-1}\left(\text{Encoder}(\pi(\mathbf{x}))\right)$$

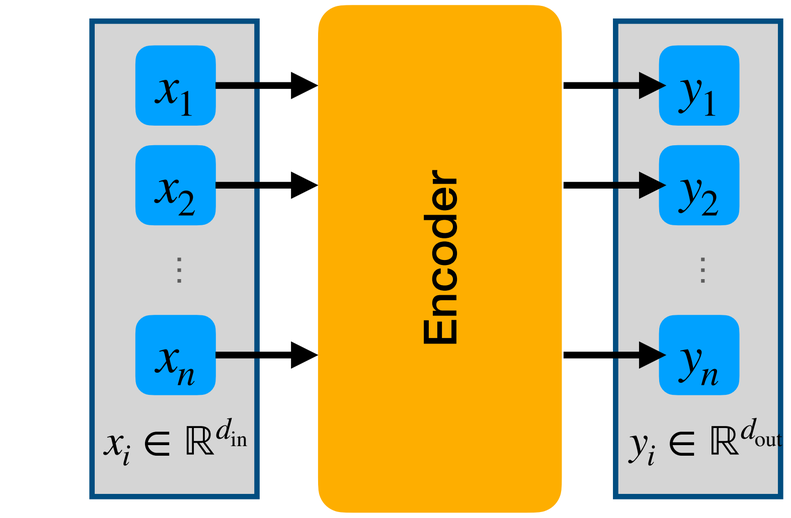
HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

Permutation-Equivariance

- We want permutation-equivariance for sets!
  $\{a, b, c\} = \{b, a, c\}$
- We don't want permutation equivariance for tuples/sequences!
  $(a, b, c) \neq (b, a, c)$

$$\text{Encoder}(\pi(\mathbf{x})) = \pi\left(\text{Encoder}(\mathbf{x})\right)$$

$$\text{Encoder}(\mathbf{x}) = \pi^{-1}\left(\text{Encoder}(\pi(\mathbf{x}))\right)$$



Multi-valued Input (Set/Sequence/Tuple)

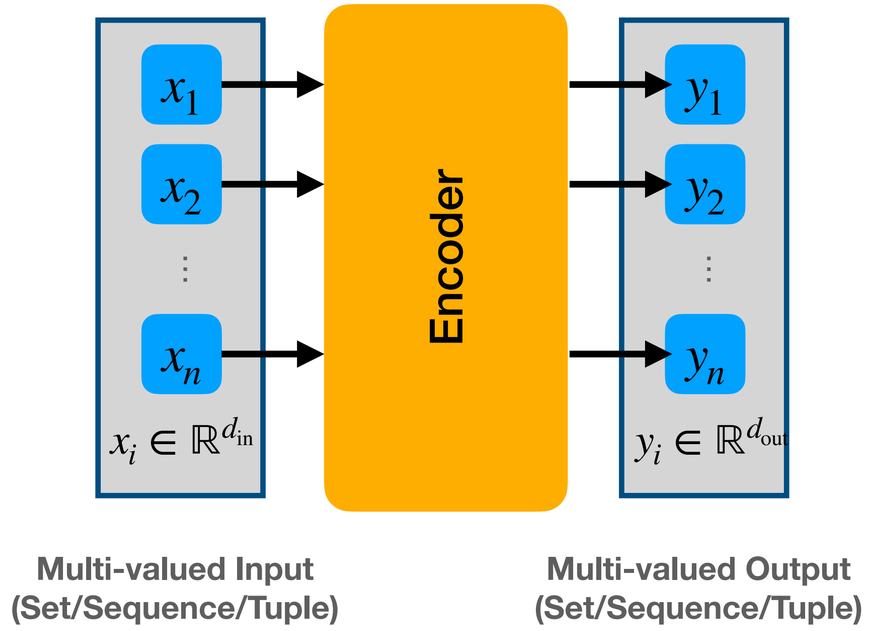
Multi-valued Output (Set/Sequence/Tuple)

HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

We don’t want permutation equivariance for tuples/sequences

BUT often we still want to keep the structure intact

- Number of input and output elements stays the same
  - each x_i and y_i are related
- Especially for sequences!



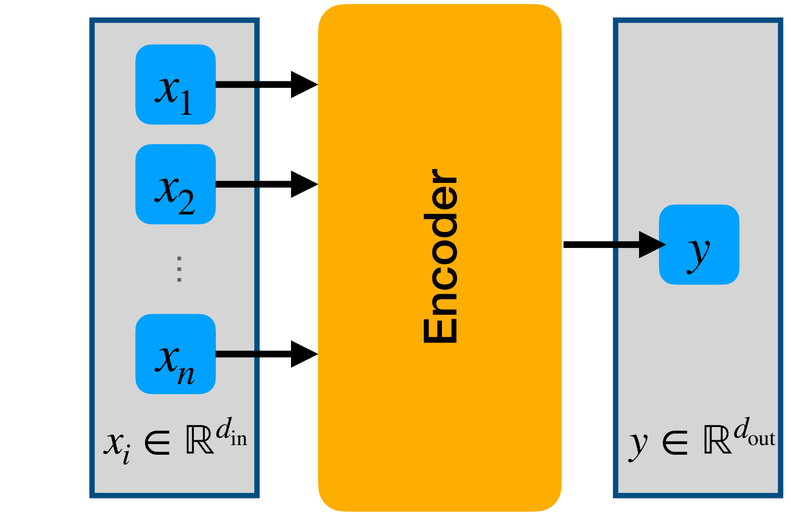
HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

Do we keep the same structure in the output y?

Sometimes we don't want any structure in the output

- Output y is then only a single element (a feature vector)
- BUT: we may still consider the structure of **x**

Aggregation Operations



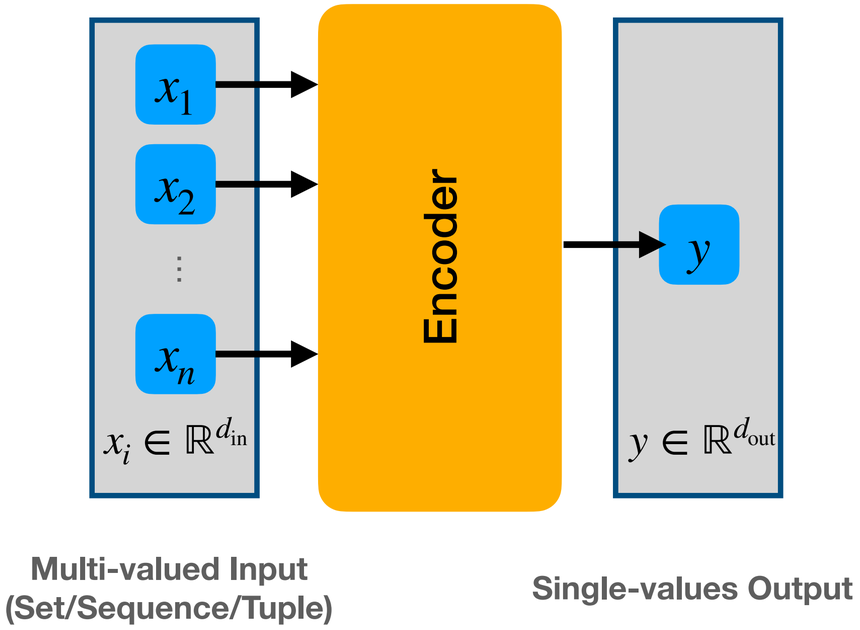
Multi-valued Input (Set/Sequence/Tuple)

Single-values Output

HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

Permutation-Invariance

- If input order changes, the output is not affected
  - Encoder must ignore order when processing features!
- Given permutation π
  - Permuted input $\mathbf{x}' = \pi(\mathbf{x})$
  - Processed original input: $y = \text{Encoder}(\mathbf{x})$
  - Processed permuted input: $y' = \text{Encoder}(\mathbf{x}')$
  - Invariance: $y' = y$



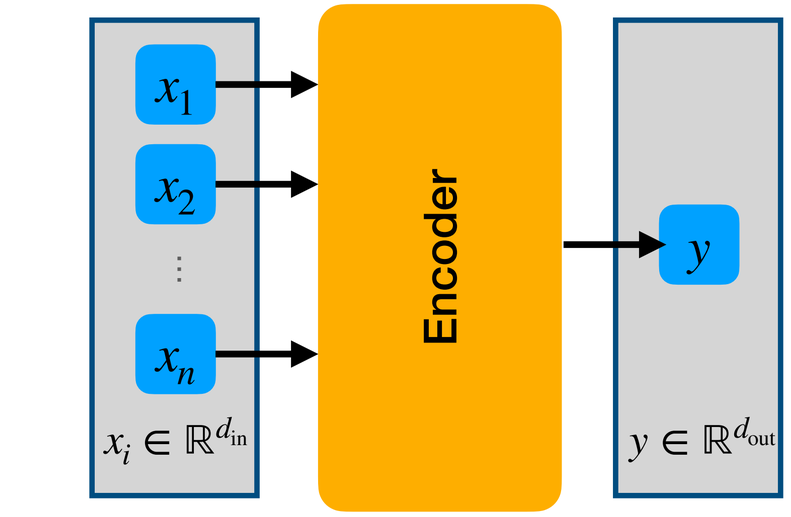
$$\text{Encoder}(\pi(\mathbf{x})) = \big(\text{Encoder}(\mathbf{x})\big)$$

HOW TO ENCODE MULTI-VALUED INPUTS? — STRUCTURE

Permutation-Invariance

- We want permutation-invariance for sets!
  $\{a, b, c\} = \{b, a, c\}$
- We don't want permutation invariance for tuples/sequences!
  $(a, b, c) \neq (b, a, c)$

BUT for sequences:
We may still want to support dynamic lengths



Multi-valued Input (Set/Sequence/Tuple)

Single-values Output

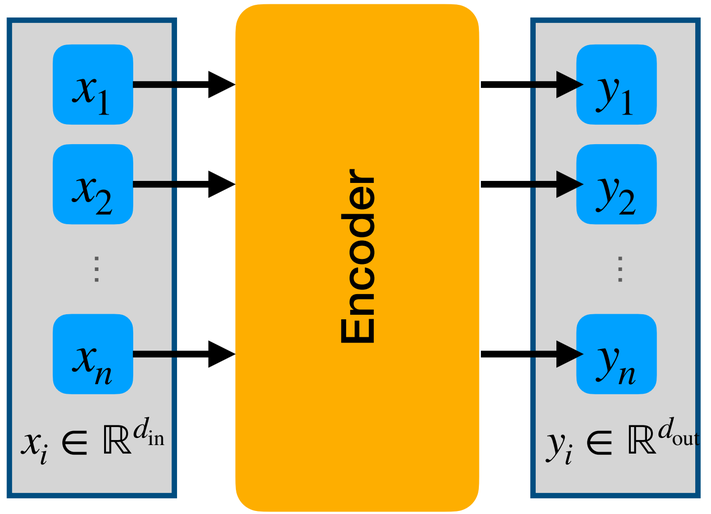
BUILDING BLOCKS FOR ENCODERS

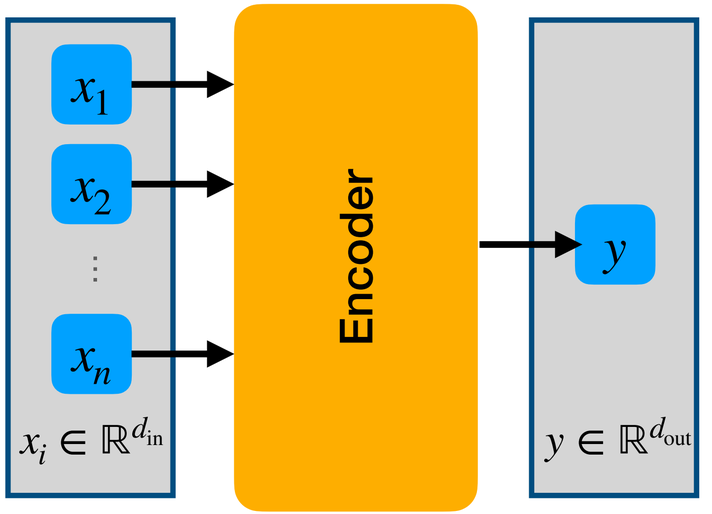
Property 1: Do they keep structure intact?

- Structure kept intact:
  - Output structure = input structure
  - Stackable operations
  - Used early in the encoder

- Structure removed:
  - No output structure (single feature vector)
  - Not stackable
  - Used at end of encoder

- Structure changes / new structure created
  - discussed later for decoders





BUILDING BLOCKS FOR ENCODERS

Property 1: Do they keep structure intact?

- **Property 1a: Permutation-equivariance / -invariance**
  - Does it support sets as input/output types?
- **Property 1b: Supporting dynamic sequences lengths**
  - Does it support sets or sequences as input/output types?

BUILDING BLOCKS FOR ENCODERS

Property 2: Parameters

- **Property 2a: Is it parameterized**
  - Only parameterized functions are trainable
- **Property 2b: Does it use parameter-sharing**
  - Parameter-sharing improves efficiency and generalization

BUILDING BLOCKS FOR ENCODERS

Property 3: Interactions

- How do different elements interact with each other
  - No interaction, pairwise, full interaction, …
- How can information flow btw elements
  - No flow, uni-/bidirectional, local/global

ELEMENT-WISE OPERATIONS

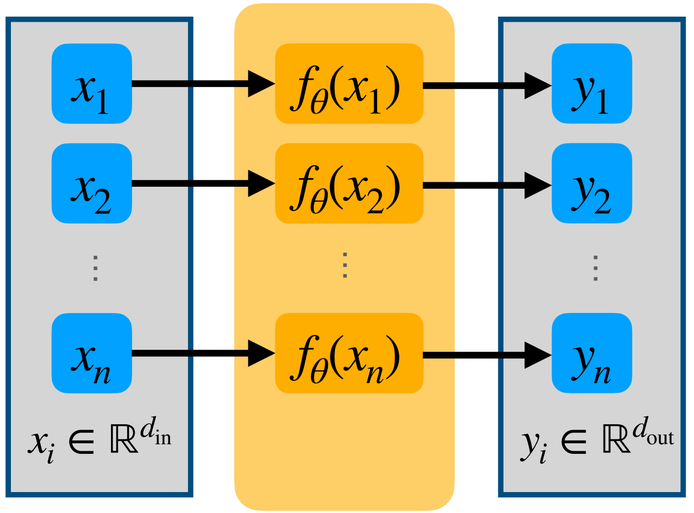
$f_\theta(x) : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$

- Input feature to output feature
- Applied to each element independently

Compute $\mathbf{y}$ from $\mathbf{x}$ as

$y_i = f_\theta(x_i)$

(repeat for each i, using the same parameters)



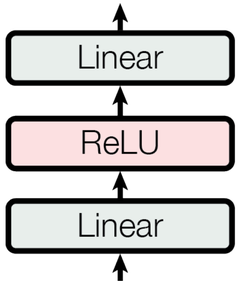
	y_1	y_2	...	y_n
x_1	1			
x_2		1		
...			1	
x_n				1

Information Flow

ELEMENT-WISE OPERATIONS — WHAT IS F(X)

MLP / FFN

- Non-linear processing of individual features
- Typically $\mathbb{R}^{d_{\text{in}}} = \mathbb{R}^{d_{\text{out}}}$



Normalization, Dropout, …

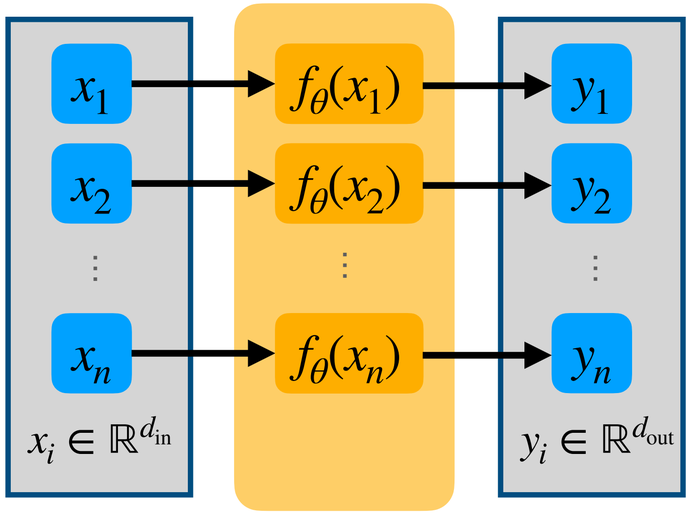
- Training stability + generalization

Residual Connections

- Training stability + deeper networks

Linear Projection

- Changing from $\mathbb{R}^{d_{\text{in}}}$ to $\mathbb{R}^{d_{\text{out}}}$



$$f_\theta(x) : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$$

ELEMENT-WISE OPERATIONS

Structure	Kept intact
Permutations	Permutation-equivariant
Lengths	Supports dynamic lengths
Parameterized	Can be, depending on used f(x) (true for MLPs)
Parameter-sharing	Yes (same function used for all elements)
Interaction	No interactions btw elements
Information flow	No information flow

How can we add information flow?

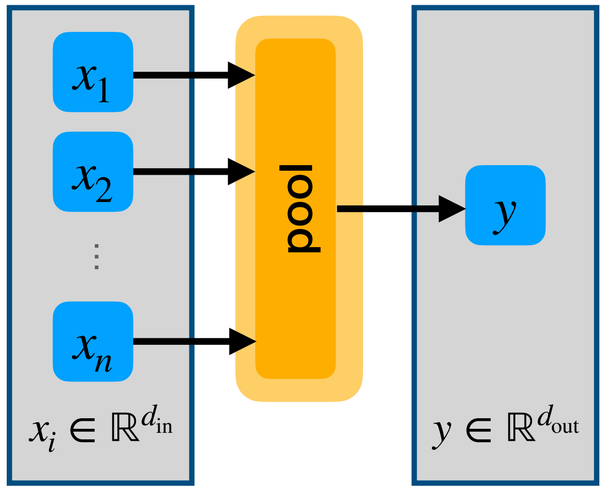
GLOBAL POOLING

Mean Pooling

$$y = \frac{1}{n} \sum_i x_i$$

Max Pooling

$$y = \max_i x_i$$



	y
x_1	1
x_2	1
...	1
x_n	1

Information Flow

GLOBAL POOLING

Structure	Removed (aggregation)
Permutations	Permutation-invariant
Lengths	Supports dynamic lengths
Parameterized	No
Parameter-sharing	(has no parameters)
Interaction	No interactions btw elements
Information flow	Global (from all alements)

What if we want to keep the structure intact?

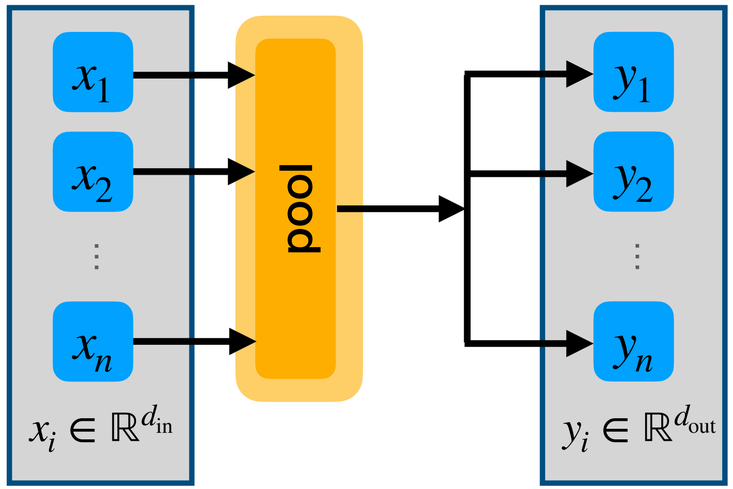
POOL AND DISTRIBUTE

Pool and Distribute

1. Pool inputs to single vector (e.g. using mean/max pooling)
2. Copy pooled vector for each element

Optional

- Apply MLP after pool / before distribute
- Add outputs to
  - residual connections of x
  - or element-wise operations on x



	y_1	y_2	...	y_n
x_1	1	1	1	1
x_2	1	1	1	1
...	1	1	1	1
x_n	1	1	1	1

Information Flow

POOL AND DISTRIBUTE

Structure	Kept intact
Permutations	Permutation-equivariant (with residual connections)
Lengths	Supports dynamic lengths
Parameterized	No
Parameter-sharing	(has no parameters)
Interaction	No interactions btw elements
Information flow	Global (from all alements)

How can we add interactions btw elements?

MLP AGGREGATION (CONCAT + MLP)

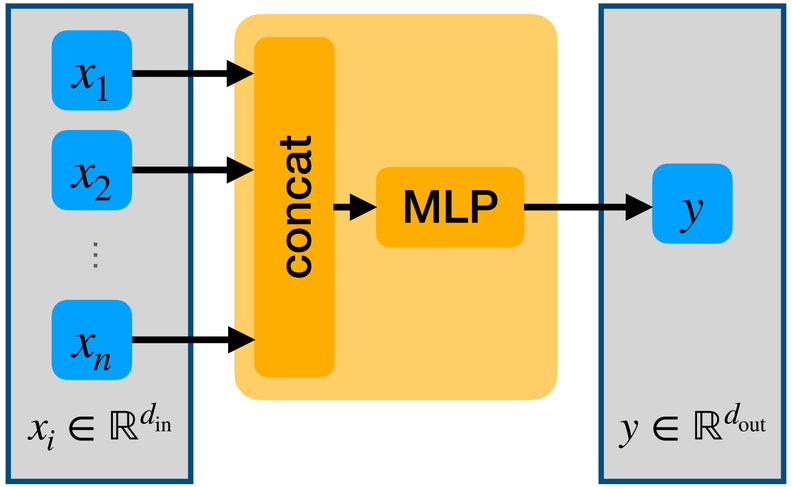
Concat + MLP

1. Concatenate elements

$$\tilde{x} = x_1 \oplus \ldots \oplus x_n = \left(x_{1,1}, \ldots x_{1,d}, \ldots, x_{n,1}, \ldots x_{n,d}\right) \in \mathbb{R}^{nd}$$

2. Apply MLP (or similar operations)

$$y = \text{MLP}\left(\tilde{x}\right) \in \mathbb{R}^{nd}$$



	y
x_1	1
x_2	1
...	1
x_n	1

Information Flow

MLP AGGREGATION (CONCAT + MLP)

Structure	Removed (aggregation)
Permutations	Not invariant
Lengths	Does not support dynamic lengths
Parameterized	Yes
Parameter-sharing	No (parameters scale linearly with input length)
Interaction	Full interaction btw elements
Information flow	Global (from all alements)

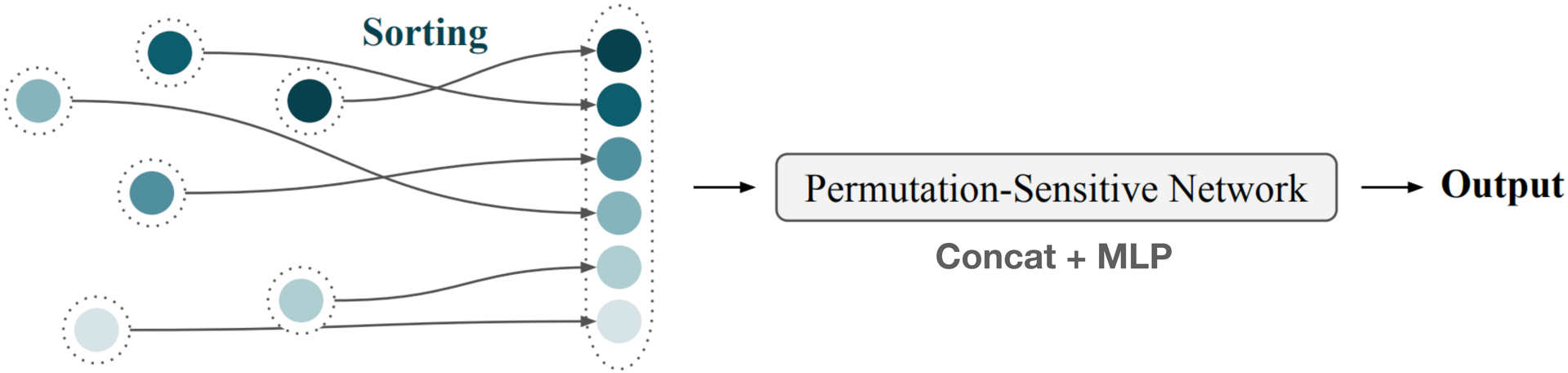
How can we make this permutation-invariant?

MLP AGGREGATION WITH SORTING

Sorting + Concat + MLP

1. Sort inputs based on element features or associated properties

2. Apply Concat + MLP



MLP AGGREGATION WITH SORTING

Structure	Removed (aggregation)
Permutations	Permutation-invariant (sorting assures original order is irrelevant)
Lengths	Does not support dynamic lengths
Parameterized	Yes
Parameter-sharing	No (parameters scale linearly with input length n)
Interaction	Full interaction btw elements
Information flow	Global (from all alements)
Problems	Sorting cost O(n log n) **How to pick canonical ordering?** Sorting not easily learned (non-differentiable)

How can we get rid of the sorting?

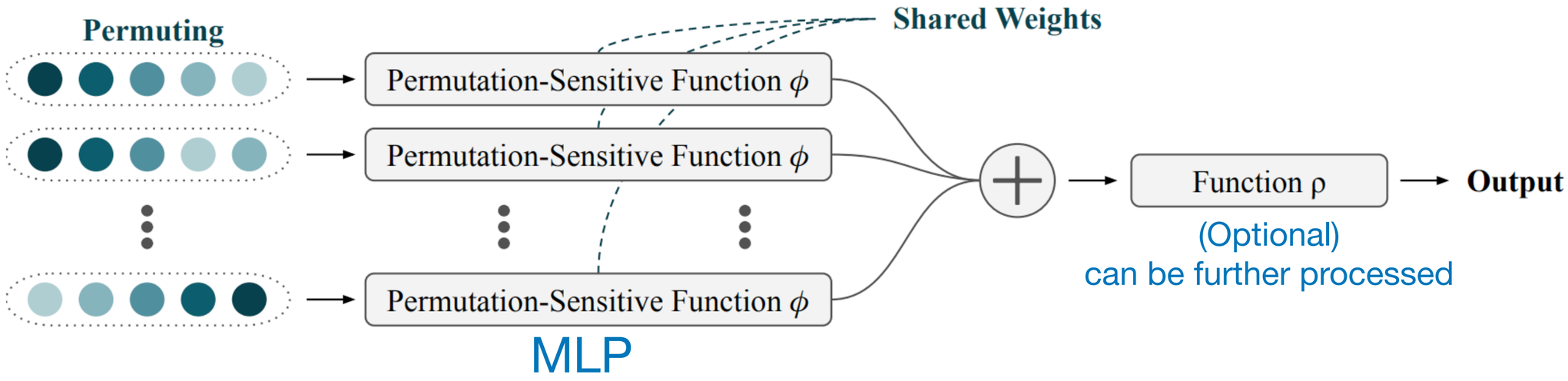
JANOSSY POOLING

Janossy Pooling (Permute + Average)

1. Build all possible permutations $\pi(x)$ of input **x**
   - $\pi \in S_n$: permutation (reorders elements in **x**)
   - S_n: set of all permutations of length n
2. Apply permutation-sensitive function ϕ (e.g. Concat + MLP)
3. Average outputs

$$y = \frac{1}{|S_n|} \sum_{\pi \in S_n} \phi_\theta(\pi(\mathbf{x}))$$

$$\phi_\theta \colon \mathbb{R}^{n \times d_{\text{in}}} \to \mathbb{R}^d$$



JANOSSY POOLING

Structure	Removed (aggregation)
Permutations	Permutation-invariant (summing over all permutations)
Lengths	Does not support dynamic lengths (MLP still applied over full length)
Parameterized	Yes
Parameter-sharing	Not really (shared over permutations but still scales with length n)
Interaction	Full interaction btw elements
Information flow	Global (from all alements)
Problems	**Computationally expensive** $\lvert S_n \rvert = n!$

How can we make this more efficient
(and support dynamic sequence lengths)?

ORDER-K JANOSSY POOLING

(Full) Janossy Pooling

- Permutes all n elements in **x**
- => Look at all possible n-tuples in **x**

$$y = \frac{1}{|S_n|} \sum_{\pi \in S_n} \phi_\theta(\pi(\mathbf{x}))$$

Parameters scale with n

$$\phi_\theta \colon \mathbb{R}^{n \times d_{\text{in}}} \to \mathbb{R}^d$$

Order-k Janossy Pooling

- Look at all possible k-tuples in **x**
  - $\pi \in S_n$: permutation
    (reorders elements in **x**)
  - S_n: set of all permutations of length n
  - $\downarrow_k (\mathbf{x}) = (x_1, \ldots, x_k)$
    projection to length k
    (take first k elements or zero-pad if $n < k$)

$$y = \frac{1}{|S_n|} \sum_{\pi \in S_n} \phi_\theta \left(\downarrow_k (\pi(\mathbf{x})) \right)$$

Parameters scale with k

$$\phi_\theta \colon \mathbb{R}^{k \times d_{\text{in}}} \to \mathbb{R}^d$$

Still $|S_n| = n!$ computations

ORDER-K JANOSSY POOLING

Order-k Janossy Pooling

- Look at all possible k-tuples in **x**
  - allows us to directly sample k-tuples
    => only $\frac{n!}{(n-k)!}$ different tuples
  - $\mathbf{x}_{\{k\}}$: a k-tuple from **x**
- Example: $\{a, b, c, d\}$
  - For $k = 2$, $\mathbf{x}_{\{2\}}$ can be one of
    $(a,b), (a,c), (a,d), (b,a), (b,c), (b,d)$
    $(c,a), (c,b), (c,d), (d,a), (d,b), (d,c)$

    $\frac{4!}{(4-2)!} = 12$ instead of $4! = 24$

$$y = \frac{1}{|S_n|} \sum_{\pi \in S_n} \phi_\theta \left(\downarrow_k \left(\pi(\mathbf{x}) \right) \right)$$

Still $|S_n| = n!$ computations
BUT: many of them are the same

$$y = \frac{(n-k)!}{n!} \sum_{\mathbf{x}_{\{k\}}} \phi_\theta \left(\mathbf{x}_{\{k\}} \right)$$

Now only $\frac{n!}{(n-k)!}$ computations,
giving same result as above

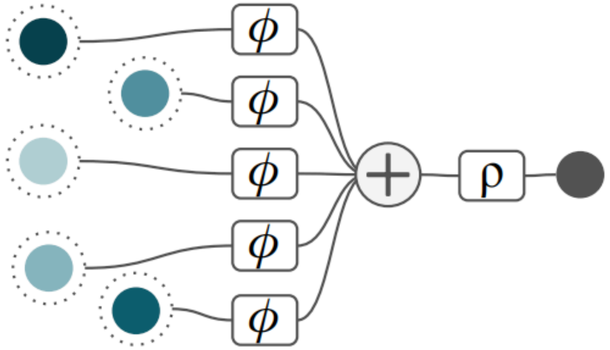
ORDER-K JANOSSY POOLING

Order-k Janossy Pooling

- Only lower-order interactions (order k)

$$\phi_\theta \colon \mathbb{R}^{k \times d_{\text{in}}} \to \mathbb{R}^d$$

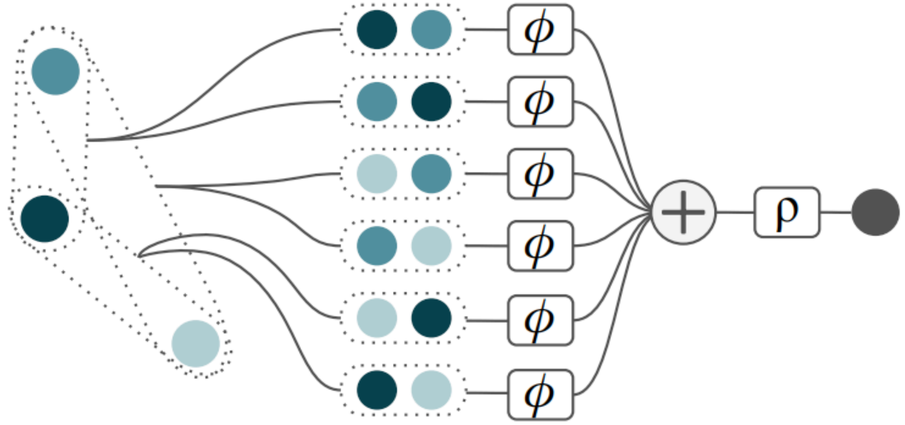
$$y = \frac{(n-k)!}{n!} \sum_{\mathbf{x}_{\{k\}}} \phi_\theta\left(\mathbf{x}_{\{k\}}\right)$$



(a) Janossy pooling with $k = 1$ (*Deep Sets*)

$$\phi_\theta \colon \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^d$$

No interactions



(b) Janossy pooling with $k = 2$

$$\phi_\theta \colon \mathbb{R}^{d_{\text{in}}} \times \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^d$$

Pairwise interactions

ORDER-K JANOSSY POOLING

Structure	Removed (aggregation)
Permutations	Permutation-invariant (summing over all k-tuples)
Lengths	Supports dynamic lengths (operation is always applied on length k-tuples)
Parameterized	Yes
Parameter-sharing	Yes (now parameters only scale with k, a hyperparameter)
Interaction	Order-k interactions btw elements (no interactions for k=1, pairwise for k=2)
Information flow	Global (from all alements)
Problems	Computationally expensive for large k BUT now controllable by hyperparam

What if we want to keep the structure intact?

EQUIVARIANT ORDER-K JANOSSY POOLING

Order-k Janossy Pooling

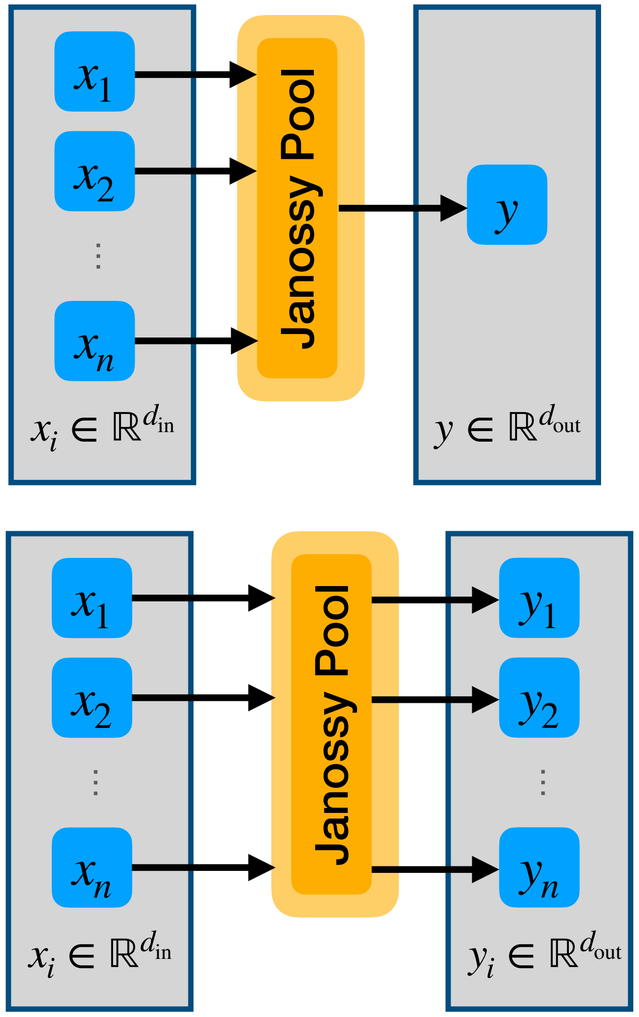
- Permutation-invariant: structure is removed
- Sums over all k-tuples

$$y = \frac{(n-k)!}{n!} \sum_{\mathbf{x}_{\{k\}}} \phi_\theta \left(\mathbf{x}_{\{k\}} \right)$$

Equivariant Order-k Janossy Pooling

- Permutation-equivariant: structure stays intact
- For each y_i, sum over all k-tuples containing x_i

$$y_i \propto \sum_{\mathbf{x}_{\{k\},i \in \{k\}}} \phi_\theta \left(\mathbf{x}_{\{k\}} \right)$$



EQUIVARIANT ORDER-K JANOSSY POOLING

Binary Janossy Pooling

- Permutation-invariant: structure is removed
- Sums over all k-tuples

$$y \propto \sum_{\mathbf{x}_{\{2\}}} \phi_\theta\left(\mathbf{x}_{\{2\}}\right) = \sum_{i,j,i\neq j} \phi_\theta\left(x_i, x_j\right)$$

Equivariant Binary Janossy Pooling

- Permutation-equivariant: structure stays intact
- For each y_i, sum over all k-tuples containing x_i

$$y_i \propto \sum_{\mathbf{x}_{\{2\},i\in\{2\}}} \phi_\theta\left(\mathbf{x}_{\{2\}}\right) = \sum_{j,i\neq j} \phi_\theta\left(x_i, x_j\right)$$

Note: $i \neq j$ is optional and could also be relaxed

Sum over full matrix (except diagonal)

$$\sum_{i,j,j\neq 1} \phi(x_i, x_j)$$

$$\begin{pmatrix} \phi\left(x_1,x_1\right) & \phi\left(x_2,x_1\right) & \dots & \phi\left(x_n,x_1\right) \\ \phi\left(x_1,x_2\right) & \phi\left(x_2,x_2\right) & \dots & \phi\left(x_n,x_2\right) \\ \vdots & & \ddots & \\ \phi\left(x_1,x_n\right) & \phi\left(x_2,x_n\right) & \dots & \phi\left(x_n,x_n\right) \end{pmatrix}$$

$$\sum_{j,j\neq 1} \phi(x_1, x_j) \quad \sum_{j,j\neq 2} \phi(x_2, x_j) \quad \sum_{j,j\neq n} \phi(x_n, x_j)$$

Sum per column (except diagonal)

EQUIVARIANT ORDER-K JANOSSY POOLING

Structure	Kept intact
Permutations	Permutation-equivariant (summing over k-tuples)
Lengths	Supports dynamic lengths (operation is always applied on length k-tuples)
Parameterized	Yes
Parameter-sharing	Yes (now parameters only scale with k, a hyperparameter)
Interaction	Order-k interactions btw elements (no interactions for k=1, pairwise for k=2)
Information flow	Global (from all alements)
Problems	Computationally expensive for large k BUT now controllable by hyperparam

What other options for $\phi_\theta\left(x_i, x_j\right)$ are there (except MLPs)?

SELF-ATTENTION VS. JANOSSY POOLING

Binary Janossy Pooling (without restriction $i \neq j$)

- $\phi(x_i, x_j)$ can be an MLP:

$$\phi(x_i, x_j) = \mathbf{W_2} \cdot \mathbf{ReLU}\left(\mathbf{W_1} \cdot (x_i \oplus x_j) + \mathbf{b_1}\right) + \mathbf{b_2}$$

$$\begin{pmatrix} \phi(x_1, x_1) & \phi(x_2, x_1) & \dots & \phi(x_n, x_1) \\ \phi(x_1, x_2) & \phi(x_2, x_2) & \dots & \phi(x_n, x_2) \\ \vdots & & \ddots & \\ \phi(x_1, x_n) & \phi(x_2, x_n) & \dots & \phi(x_n, x_n) \end{pmatrix}$$

$$\sum_j \phi(x_1, x_j) \quad \sum_j \phi(x_2, x_j) \quad \sum_j \phi(x_n, x_j)$$

Sum per column

Self-Attention

- Decompose $\phi(x_i, x_j)$ into
  - weight function $w(x_i, x_j)\colon \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$
  - and value function $v(x_j)\colon \mathbb{R}^d \to \mathbb{R}^d$

$$\phi(x_i, x_j) = w(x_i, x_j) \cdot v(x_j)$$

$$\begin{pmatrix} w(x_1, x_1) \cdot v(x_1) & w(x_2, x_1) \cdot v(x_1) & \dots & w(x_n, x_1) \cdot v(x_1) \\ w(x_1, x_2) \cdot v(x_1) & w(x_2, x_2) \cdot v(x_2) & \dots & w(x_n, x_2) \cdot v(x_2) \\ \vdots & & \ddots & \\ w(x_1, x_n) \cdot v(x_n) & w(x_2, x_n) \cdot v(x_n) & \dots & w(x_n, x_n) \cdot v(x_n) \end{pmatrix}$$

$$\sum_j w(x_1, x_j) \cdot v(x_j) \quad \sum_j w(x_2, x_j) \cdot v(x_j) \quad \sum_j w(x_n, x_j) \cdot v(x_j)$$

Sum per column

SELF-ATTENTION VS. JANOSSY POOLING

Self-Attention

- Decompose $\phi(x_i, x_j)$ into
  - weight function $w(x_i, x_j) \colon \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$
  - and value projection $v(x_j) \colon \mathbb{R}^d \to \mathbb{R}^d$

$$\phi(x_i, x_j) = w(x_i, x_j) \cdot v(x_j)$$

$$\begin{pmatrix} w(x_1,x_1)\cdot v(x_1) & w(x_2,x_1)\cdot v(x_1) & \dots & w(x_n,x_1)\cdot v(x_1) \\ w(x_1,x_2)\cdot v(x_1) & w(x_2,x_2)\cdot v(x_2) & \dots & w(x_n,x_2)\cdot v(x_2) \\ \vdots & & \ddots & \\ w(x_1,x_n)\cdot v(x_n) & w(x_2,x_n)\cdot v(x_n) & \dots & w(x_n,x_n)\cdot v(x_n) \end{pmatrix}$$

$$\sum_j w(x_1,x_j)\cdot v(x_j) \qquad \sum_j w(x_2,x_j)\cdot v(x_j) \qquad \sum_j w(x_n,x_j)\cdot v(x_j)$$

Sum per column

$$y_i = \frac{1}{n}\sum_j \phi(x_i, x_j) = \frac{1}{n}\sum_j w(x_i, x_j) v(x_j)$$

Problem: $\sum_j w(x_i, x_j) = 1$ cannot beguaranteed

SELF-ATTENTION VS. JANOSSY POOLING

Self-Attention

- Decompose $\phi(x_i, x_j)$ into
  - weight function $w(x_i, x_j) : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$
  - and value function $v(x_j) : \mathbb{R}^d \to \mathbb{R}^d$
- Normalize weights before summing (softmax)

$$y_i = \frac{1}{n} \sum_j w(x_i, x_j) v(x_j)$$

$$p_{i,j} = \text{softmax}_j \left(w(x_i, x_j) \right) = \sum_j \frac{\exp w(x_i, x_j)}{\sum_{j'} \exp w(x_i, x_{j'})}$$

$$y_i = \sum_j p_{i,j} \cdot v(x_j) = \sum_j \text{softmax}_j \left(w(x_i, \mathbf{x}) \right) v(x_j)$$

Now: $\sum_j \tilde{w}_{i,j} = 1$ => weighted average

SELF-ATTENTION VS. JANOSSY POOLING

Self-Attention

- Each single normalized weight $\tilde{w}_{i,j}$ depend on the full column (sum over j')

$$p_{i,j} = \sum_j \frac{\exp w(x_i, x_j)}{\sum_{j'} \exp w(x_i, x_{j'})}$$

$$\begin{pmatrix} p_{1,1} \cdot v(x_1) & p_{2,1} \cdot v(x_1) & \dots & p_{n,1} \cdot v(x_1) \\ p_{1,2} \cdot v(x_1) & p_{2,2} \cdot v(x_2) & \dots & p_{n,2} \cdot v(x_2) \\ \vdots & & \ddots & \\ p_{1,n} \cdot v(x_n) & p_{2,n} \cdot v(x_n) & \dots & p_{n,n} \cdot v(x_n) \end{pmatrix}$$

=> this cannot be represented as binary Janossy Pooling

=> **BUT:** it is very similar (binary interactions)

SELF-ATTENTION — WEIGHT FUNCTION

What is the weight function $w(x_i, x_j) : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$

- Could be any similarity or distance, even an MLP is possible

Scaled Dot Product Attention (most common form)

dot product

$$w(x_i, x_j) = \frac{x_i^T x_j}{\sqrt{d}}$$

- Add query / key projections:

$$w(x_i, x_j) = \frac{q(x_i)^T k(x_j)}{\sqrt{d}}$$

SELF-ATTENTION — SCALED DOT PRODUCT ATTENTION

- query projection $q(x_i): \mathbb{R}^d \to \mathbb{R}^d$ (linear projection)
- key projection $k(x_j): \mathbb{R}^d \to \mathbb{R}^d$ (linear projection)
- value projection $v(x_j): \mathbb{R}^d \to \mathbb{R}^d$ (linear projection)
- weight function: scaled dot product $w(x_i, x_j) = \dfrac{q(x_i) \cdot k(x_j)}{\sqrt{d}}$
- attention scores / probabilities:
  softmax distribution over keys

$$p_{i,j} = \text{softmax}_j\left(w(x_i, x_j)\right) = \text{softmax}_j\left(\frac{q(x_i)^T k(\mathbf{x})}{\sqrt{d}}\right) \qquad \sum_j p_{i,k} = 1 \quad \forall i$$

- self-attention for query i:

$$y_i = \sum_j p_{i,j} \cdot v(x_j) = \sum_j \text{softmax}_j\left(\frac{q(x_i)^T k(\mathbf{x})}{\sqrt{d}}\right) v(x_j)$$

SELF-ATTENTION — MULTI-HEAD ATTENTION

Problem:

Only one attention distribution $p_{i,j}$ for each query i

BUT:

There may be different aspects based on which we want to aggregate

Solution:

Apply self-attention multiple times with different query/key/value projections

SELF-ATTENTION — MULTI-HEAD ATTENTION

- Independent **linear** query-key-value projections for each head h:

  $q_h(x_i) : \mathbb{R}^d \to \mathbb{R}^{d_{\text{head}}}, \quad k_h(x_j) : \mathbb{R}^d \to \mathbb{R}^{d_{\text{head}}}, \quad v_h(x_j) : \mathbb{R}^d \to \mathbb{R}^{d_{\text{head}}}$

- Scaled dot product attention on smaller head dimension d_{head},

  typically $d_{\text{head}} = \left\lfloor \frac{d}{h} \right\rfloor$

- Compute outputs per head

$$\tilde{y}_{h,i} = \sum_j p_{h,i,j} \cdot v_h(x_j) = \sum_j \text{softmax}_j \left(\frac{q_h(x_i)^T k_h(\mathbf{x})}{\sqrt{d}} \right) v(x_j)$$

- Combine by concatenation and **linear projection** $o\left(\tilde{\mathbf{y}}\right) : \mathbb{R}^{h \cdot d_{\text{head}}} \to \mathbb{R}^d$ of per-head outputs

$$y_i = o\left(\text{concat}_h\left(\tilde{y}_{h,i}\right) \right)$$

(MULTIHEAD) SELF-ATTENTION

Structure	Kept intact
Permutations	Permutation-equivariant (output order based on query order, which follows input order)
Lengths	Supports dynamic lengths (one output per query, which follows input)
Parameterized	Yes (query, key, value, output projections)
Parameter-sharing	Yes (same projections are applied for each element)
Interaction	Pairwise interactions during attention computation (no explicit interactions between values)
Information flow	Global (from all alements)
Problems	Quadratic complexicity w.r.t. input length

Can we also use attention for aggregation?

QUERY-KEY-VALUE (QKV) ATTENTION

Query-Key-Value Attention

- Generalized self-attention
  - Think in terms of queries, keys, and values
  - **They do not have to come from the same source!**
- Matrix notation:
  - query matrix $\mathbf{Q} \in \mathbb{R}^{n_\mathrm{q} \times d_\mathrm{q}}$
  - key matrix $\mathbf{K} \in \mathbb{R}^{n_\mathrm{v} \times d_\mathrm{q}}$ (d like queries, n like values)
  - value matrix $\mathbf{V} \in \mathbb{R}^{n_\mathrm{v} \times d_\mathrm{v}}$ (d may differ)
  - weight matrix (attention scores) $\mathbf{S} \in \mathbb{R}^{n_\mathrm{q} \times n_\mathrm{v}}$
  - attention probabilities $\mathbf{P} \in [0,1]^{n_\mathrm{q} \times n_\mathrm{v}}, \forall i \sum_j P_{i,j} = 1$
  - output (per head) $\mathbf{O} \in \mathbb{R}^{n_\mathrm{q} \times d_\mathrm{v}}$

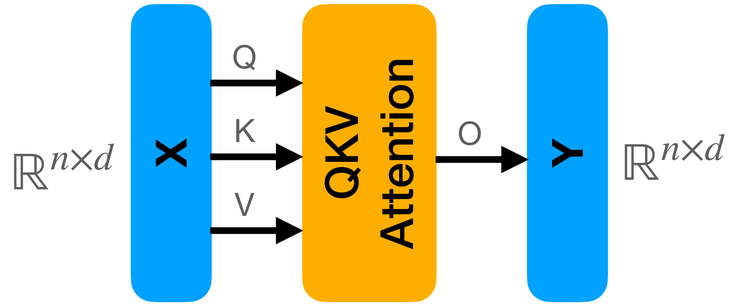
$$\mathbf{S} = \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} \right) \qquad \mathbf{P} = \mathrm{softmax}\left(\mathbf{S}\right)$$

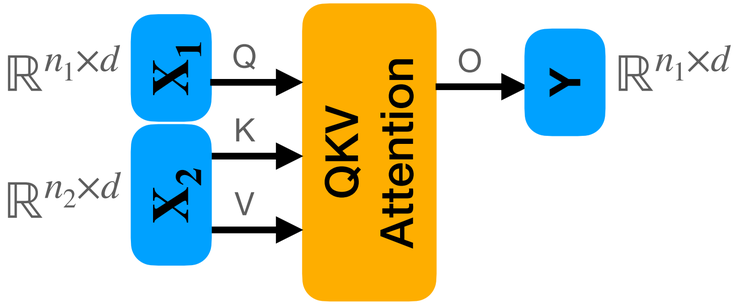
$$\mathbf{O} = \mathbf{P}\mathbf{V}$$

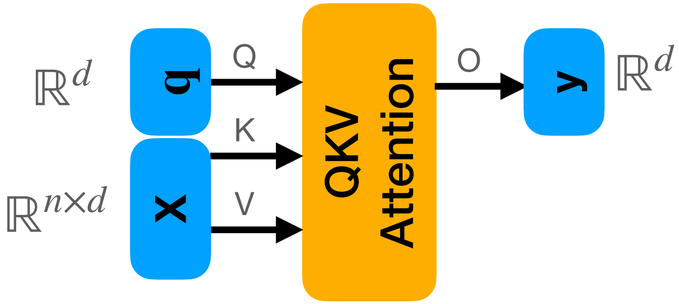
$$\mathbf{O} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} \right) \mathbf{V}$$

QUERY-KEY-VALUE (QKV) ATTENTION

- **Self-attention**
  - Query, key, and value matrix from same source
  - Input and output length stay the same
  - Stackable operation
- **Cross-attention**
  - Query matrix from one source
  - Key/value matrix come from different source
  - Output length = query length
  - Condition query set on key/value set
- **Attention-pooling**
  - Single query (query matrix becomes query vector)
  - Key and value matrix come from the set to pool over
  - Single output element
  - Aggregates over a set







AGGREGATION WITH ATTENTION — ATTENTION POOLING

Attention Pooling: where does the query vector come from

- Learned parameter
  - Constant over all samples
  - There may be several independent queries to get aggregations for different aspects (e.g. one query per target class)
- Computed using other aggregation on input set
  - E.g. first use global average pooling to get query vector
  - Then use attention-pooling to selectively aggregate input

(MULTIHEAD) CROSS-ATTENTION

Structure	Query structure kept in output
Permutations	Permutation-equivariant (wrt. both query and key/value set)
Lengths	Supports dynamic lengths (for both query and key/value set)
Parameterized	Yes (query, key, value, output projections)
Parameter-sharing	Yes (same projections are applied for each element)
Interaction	Pairwise interactions btw. queries and keys (no explicit interactions between values)
Information flow	Global from key/value set (No flow between query elements)
Problems	Linear complexicity wrt. both query and key/value set

(MULTIHEAD) ATTENTION-POOLING

Structure	Removed (aggregation)
Permutations	Permutation-invariant
Lengths	Supports dynamic lengths
Parameterized	Yes (query, key, value, output projections)
Parameter-sharing	Yes (same projections are applied for each element)
Interaction	Pairwise interactions btw. query and keys (no explicit interactions between values)
Information flow	Global from key/value set
Problems	How to compute the query

Is (full) attention the perfect solution?

QUADRATIC COMPLEXITY OF ATTENTION

Computing Self-attention on set of size n has time (and space) complexity $\mathcal{O}\left(n^2\right)$

- Each element acts as a query i (linear in n)
- Output for each query i sums over all j (linear in n)
  - keys (to compute softmax)
  - and values (to compute final output)
- Due to softmax this cannot be simplified!

$$\begin{matrix} \sum_j & \sum_j & & \sum_j \\ \begin{pmatrix} p_{1,1} \cdot v(x_1) & p_{2,1} \cdot v(x_1) & \cdots & p_{n,1} \cdot v(x_1) \\ p_{1,2} \cdot v(x_1) & p_{2,2} \cdot v(x_2) & \cdots & p_{n,2} \cdot v(x_2) \\ \vdots & & \ddots & \\ p_{1,n} \cdot v(x_n) & p_{2,n} \cdot v(x_n) & \cdots & p_{n,n} \cdot v(x_n) \end{pmatrix} \end{matrix}$$

$$p_{i,j} = \text{softmax}_j\left(w(x_i, x_j)\right) = \text{softmax}_j\left(\frac{q(x_i)^T k(\mathbf{x})}{\sqrt{d}}\right)$$

$$y_i = \sum_j p_{i,j} \cdot v(x_j) = \sum_j \text{softmax}_j\left(\frac{q(x_i)^T k(\mathbf{x})}{\sqrt{d}}\right) v(x_j)$$

QUADRATIC COMPLEXITY OF ATTENTION

Potential Solutions

- Restricting attention patterns
- Alternative attention formulations: linear attention
- Causal interactions + memory
- Ignore problem, but optimize for hardware

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

Restricting attention patterns

- Only compute a subset of interactions

➡ Set some elements in $p_{i,j}$ to 0

$$\begin{pmatrix} \sum_j & \sum_j & & \sum_j \\ p_{1,1} \cdot v(x_1) & p_{2,1} \cdot v(x_1) & \cdots & p_{n,1} \cdot v(x_1) \\ p_{1,2} \cdot v(x_1) & p_{2,2} \cdot v(x_2) & \cdots & p_{n,2} \cdot v(x_2) \\ \vdots & & \ddots & \\ p_{1,n} \cdot v(x_n) & p_{2,n} \cdot v(x_n) & \cdots & p_{n,n} \cdot v(x_n) \end{pmatrix}$$

Example with 3 elements:

$$\begin{pmatrix} p_{1,1} \cdot v(x_1) & p_{2,1} \cdot v(x_1) & p_{3,1} \cdot v(x_1) \\ p_{1,2} \cdot v(x_1) & p_{2,2} \cdot v(x_2) & p_{3,2} \cdot v(x_2) \\ p_{1,3} \cdot v(x_3) & p_{2,3} \cdot v(x_3) & p_{3,3} \cdot v(x_3) \end{pmatrix}$$

$$\forall i, j \colon p_{i,j} > 0$$

$$\begin{pmatrix} p_{1,1} \cdot v(x_1) & 0 & 0 \\ p_{1,2} \cdot v(x_1) & p_{2,2} \cdot v(x_2) & 0 \\ 0 & p_{2,3} \cdot v(x_3) & p_{3,3} \cdot v(x_3) \end{pmatrix}$$

$$\exists i, j \colon p_{i,j} = 0$$

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

Restricting attention patterns

- Only compute a subset of interactions

➡ Set some elements in $p_{i,j}$ to 0

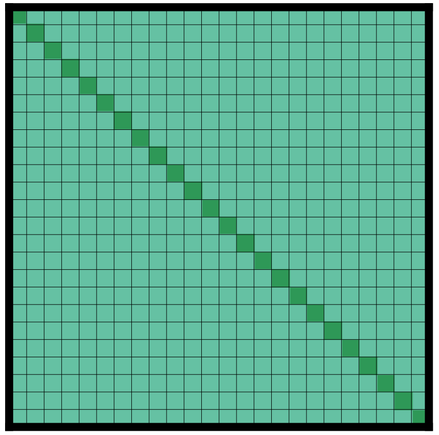
(actually set some $w(x_i, x_j) = -\infty$ before softmax, which leads to $p_{i,j} = 0$)

➡ Find a pattern that reduces the complexity

Number of operations: $\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} \leq n^2$

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

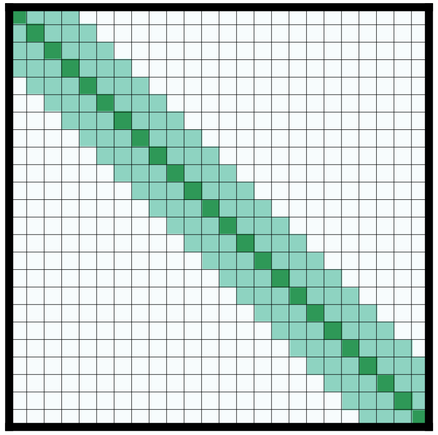
Restricting attention patterns



full attention (default)
unrestricted

$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} = n^2$$

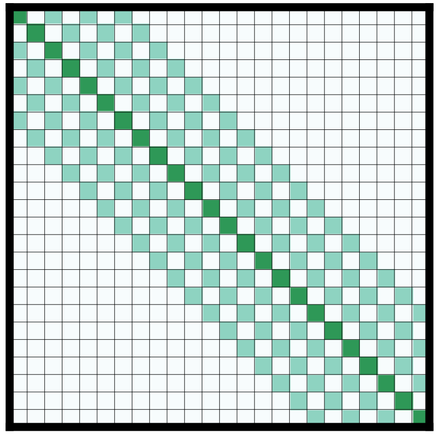
$$\Rightarrow \mathcal{O}(n^2)$$



sliding window
$|i-j| \leq k$

$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} < n \cdot 2k+1$$

$$\Rightarrow \mathcal{O}(n),$$
k is a constant



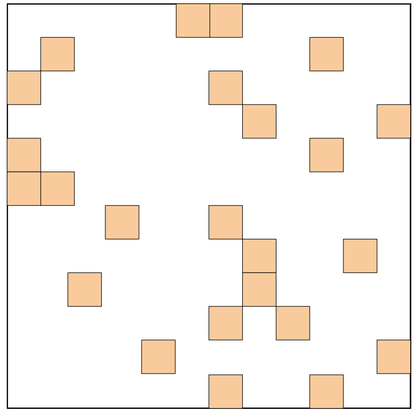
dilated sliding window
$|i-j| \leq k \cdot d$

$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} < n \cdot 2k+1$$

$$\Rightarrow \mathcal{O}(n),$$
k is a constant

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

Restricting attention patterns



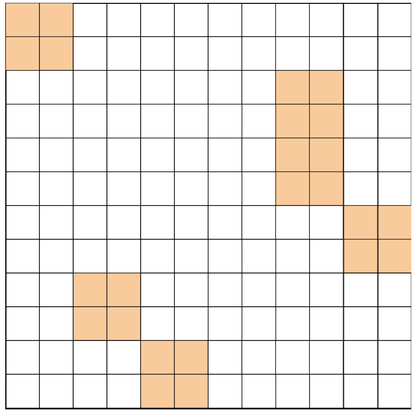
random

sample r random cells

$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} = r$$

➡ $\mathcal{O}(1)$,

r is a constant



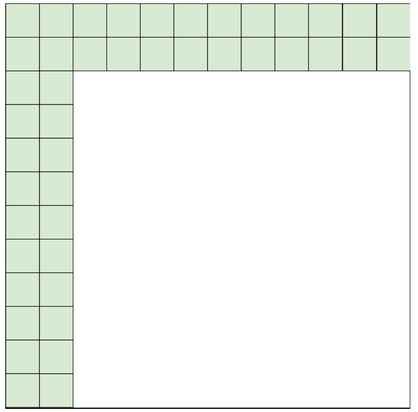
random blocks

sample r random blocks of size s

$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} = r \cdot s^2$$

➡ $\mathcal{O}(1)$,

r, s is are constant



global

g fixed tokens attending to all tokens

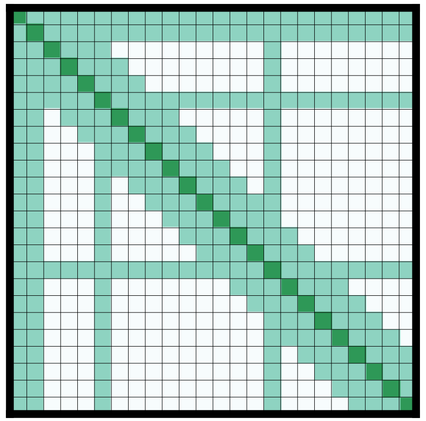
$$\sum_{i,j} \mathbf{1}_{[p_{i,j}>0]} = g \cdot n + g^2$$

➡ $\mathcal{O}(n)$,

g is a constant

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

Restricting attention patterns



Longformer

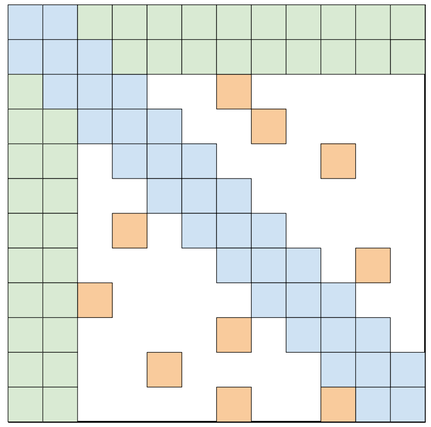
sliding window + global

Beltagy et al. (2020)

$$\max_i \sum_j \mathbf{1}_{[p_{i,j}>0]} < n \cdot (2k + 1 + g) + g^2$$

➡ $\mathcal{O}(n)$,

k, g are a constant



BigBird

sliding window + global + random

Zaheer et al. (2020)

$$\max_i \sum_j \mathbf{1}_{[p_{i,j}>0]} n \cdot (2k + 1 + g) + g^2 + r$$

➡ $\mathcal{O}(n)$,

k, g, r are a constant

QUADRATIC COMPLEXITY OF ATTENTION: ATTENTION PATTERNS

Restricting attention patterns

- Only compute a subset of interactions

➡ Set some elements in $p_{i,j}$ to 0

➡ Find a pattern that reduces the complexity

➡ **Linear complexity is achievable**

➡ **Problem: no or very restricted (global/random) long-term context**

Note: many restrictions assume ordering (sequences) / are not applicable for sets

QUADRATIC COMPLEXITY OF ATTENTION: LINEAR ATTENTION

Alternative attention formulations: Linear attention

➡ Rewrite attention without softmax

Softmax Self-attention (default):

$$y_i = \sum_j \text{softmax}_j\left(\frac{q(x_i)^T k(\mathbf{x})}{\sqrt{d}}\right) v(x_j) = \sum_j \frac{\exp\left(q(x_i)^T k(x_j)/\sqrt{d}\right)}{\sum_{j'} \exp\left(q(x_i)^T k(x_{j'})/\sqrt{d}\right)} v(x_j)$$

$$= \frac{\sum_j \text{sim}\left(q(x_i), k(x_j)\right) v(x_j)}{\sum_j \text{sim}\left(q(x_i), k(x_j)\right)} \qquad \text{with } \text{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{d}}\right)$$

QUADRATIC COMPLEXITY OF ATTENTION: LINEAR ATTENTION

Alternative attention formulations: Linear attention

➡ Rewrite attention without softmax

Softmax Self-attention (default):

$$y_i = \frac{\sum_j \text{sim}\left(q(x_i), k(x_j)\right) v(x_j)}{\sum_j \text{sim}\left(q(x_i), k(x_j)\right)} \qquad \text{with } \text{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{d}}\right)$$

Linearized attention (kernel-based):

Define $\text{sim}(q,k)$ **as a kernel** $k(q,k) = \varphi(q)^T \varphi(k)$**:**

where φ is a feature transformation (a choice leading to a specific kernel $k(q,k)$)

$$y_i = \frac{\sum_j \text{sim}\left(q(x_i), k(x_j)\right) v(x_j)}{\sum_j \text{sim}\left(q(x_i), k(x_j)\right)} = \frac{\sum_j \varphi\left(q(x_i)\right)^T \varphi\left(k(x_j)\right) v(x_j)}{\sum_j \varphi\left(q(x_i)\right)^T \varphi\left(k(x_j)\right)} = \frac{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right) v(x_j)^T}{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right)}$$

QUADRATIC COMPLEXITY OF ATTENTION: LINEAR ATTENTION

Alternative attention formulations: Linear attention

➡ Rewrite attention without softmax

Softmax Self-attention (default):

sim depends on i and j => quadratic complexity $\mathcal{O}(n^2)$

$$y_i = \frac{\sum_j \text{sim}\left(q(x_i), k(x_j)\right) v(x_j)}{\sum_j \text{sim}\left(q(x_i), k(x_j)\right)} \qquad \text{with } \text{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{d}}\right)$$

Linearized attention (kernel-based):

$$y_i = \frac{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right) v(x_j)^T}{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right)}$$

after simplification:

- $\varphi\left(q(x_i)\right)$ depends only on i
- $\sum_j \varphi\left(k(x_j)\right) v(x_j)^T$ depends only on j

➡ each linear in n => $\mathcal{O}(n)$

QUADRATIC COMPLEXITY OF ATTENTION: LINEAR ATTENTION

Alternative attention formulations: Linear attention

➡ Rewrite attention without softmax

Linearized attention (kernel-based):

BUT: $\text{sim}(q,k) = \exp\left(\frac{q^T k}{\sqrt{d}}\right)$ can also be expressed as a kernel (exponential kernel)

Why does the simplification not work for softmax attention?

$$y_i = \frac{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right) v(x_j)^T}{\varphi\left(q(x_i)\right)^T \sum_j \varphi\left(k(x_j)\right)}$$

Problem: For exponential kernel, φ maps to infinite dimension space
=> simplification is not computable

QUADRATIC COMPLEXITY OF ATTENTION: LINEAR ATTENTION

Alternative attention formulations: Linear attention

- Kernel / feature map choice can differ (e.g. polynomial kernels)
- Further readings (not relevant for this lecture):
  - https://arxiv.org/pdf/2006.16236
  - https://arxiv.org/pdf/1908.11775
  - https://arxiv.org/abs/2405.16605
  - also related: Mamba (https://arxiv.org/abs/2312.00752)

➡ **Problem:**

Data is compressed into a matrix-valued state $\sum_j \varphi\left(k(x_j)\right) v(x_j)^T$

➡ **Lower expressiveness of interactions / no real full interactions**

QUADRATIC COMPLEXITY OF ATTENTION: CAUSAL WITH MEMORY

Causal modeling + memory

- Assume causal dependencies
  (sequences with ordering, each element can only attend to previous elements)

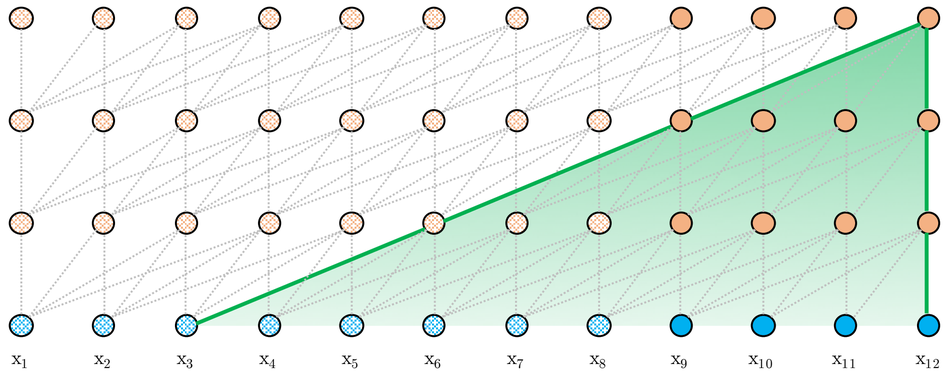
➡ $p_{i,j}$ is now a triangular matrix, which is still $\mathcal{O}(n^2)$

➡ **BUT:** we can combine it with sliding windows => $\mathcal{O}(n)$

- Sliding window is our current context (attention only within context)
- Previous states depend on even earlier states

➡ Transitive path to many more previous steps, beyond the context length

	y_1	y_2	...	y_n
x_1	1	1	1	1
x_2		1	1	1
...			1	1
x_n				1

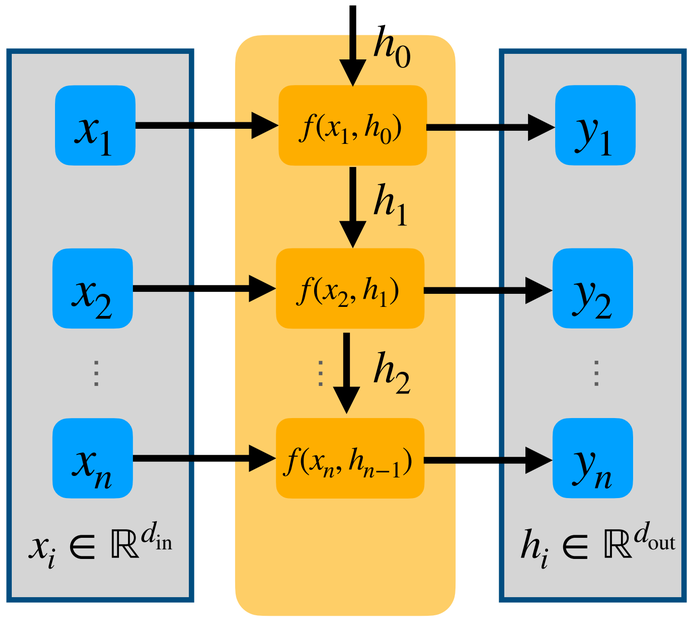


QUICK EXCURSE: RECURSIVE NEURAL NETWORKS (RNNS)

- Apply shared function $f(x_i, h_{i-1})$ recursively
  - depends on input x_i
  - depends on previous state h_{i-1}
  - returns output y_i
  - return new state h_i

➡ Consecutively update state h_i

➡ State h_i is the **memory** of the previous inputs



QUADRATIC COMPLEXITY OF ATTENTION: CAUSAL WITH MEMORY

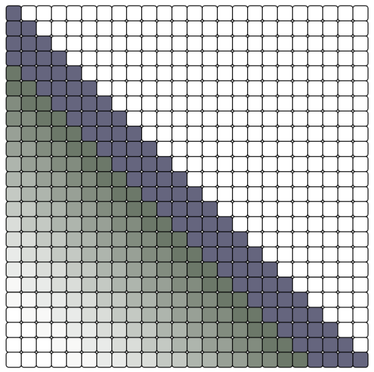
Causal modeling + memory

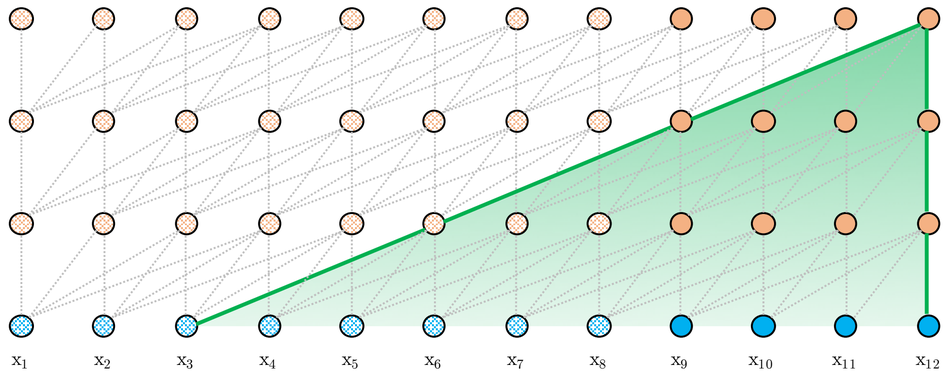
➡ Short-term memory: (causal) self attention

- full access to previous steps in context (window) of size w
- output of layer h / step i directly depends on outputs of layer $h-1$ / steps $j > i - w$

➡ Long-term memory: information stored in previous values

- transitive access beyond context, using information in outputs of previous layer
- output of layer h / step i transitively depends on outputs of steps $j > i - w \cdot (h-1)$
- each layer adds extends the receptive field by the context size





QUADRATIC COMPLEXITY OF ATTENTION: CAUSAL WITH MEMORY

Causal modeling + memory

- Causal dependencies with sliding windows

➡ Long-term memory: information stored in previous values

Problems:

- **Transitive access through limited states limits long-term dependencies**
- **No gradients or explicit training to enforce effective storage in memory**

QUADRATIC COMPLEXITY OF ATTENTION: CAUSAL WITH MEMORY

Causal modeling + memory

Problems:

- **Transitive access through limited states limits long-term dependencies**
- **No gradients or explicit training to enforce effective storage in memory**

Solutions:

- Simple but effective: Summarize conversations from time to time
- More sophisticated: Explicitly manage and compress a long-term memory
  - e.g. Titans: keep neural memory through inference-time training https://arxiv.org/pdf/2501.00663

Note: causal modeling assumes ordering (sequences) / is not applicable for sets

QUADRATIC COMPLEXITY OF ATTENTION: HARDWARE OPTIMIZATIONS

Ignore problem, but optimize for hardware

- Optimize attention computation for GPU hardware architecture
- e.g. recompute some parts because memory access can be slower than recompute
- see https://arxiv.org/abs/2205.14135 for further reading (not relevant for this lecture)

➡ **Integrated in many modern frameworks (e.g. Pytorch)**

➡ **Today's implementations and hardware support huge sequence lengths, even with full attention**

SUMMARY OF ENCODER COMPONENTS

- Different input types (sets vs. sequences vs. tuples) require different components
  - Depending on what properties you want to preserve
- Components differ in types of interactions they support and in their efficiency
- Some components are strictly superior than other ones while other simply provide different trade-offs
  - We'll later look at real architectures to understand what is currently used and why

=> There is no one size fits all solution!

BUT: is there something that fits for most cases?

Everything

is

a

Set

ENCODING STRUCTURE AS FEATURES

Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

Example: Sequences as sets

- Positions / indices are stored inside the elements
- Set of pairs of data and position

$$\left(a_i\right)_{i \leq n} = \left\{ \left(a_1, 1\right), \ldots, \left(a_n, n\right) \right\}$$

SEQUENCES AS SETS: POSITIONAL ENCODINGS

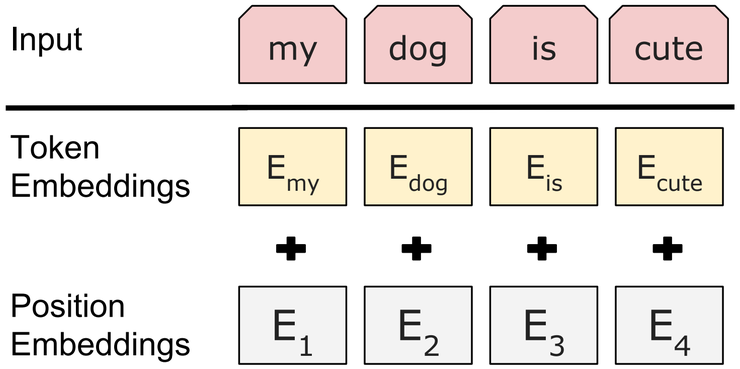
Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

$$(a_i)_{i \leq n} = \{(a_1, 1), \ldots, (a_n, n)\}$$

In practice: Positional Encodings

- Input embeddings = Sum of:
  - Data embeddings (e.g. word tokens)
  - Position embeddings (based on position in sequence)
- Position embeddings
  - Vector per element, same size as data embedding
  - Learned or hard-coded (e.g. sinusoidal)



SEQUENCES AS SETS: RELATIVE POSITIONAL ENCODINGS

What about very long sequences?

- Large positions are much rarer than smaller ones
- Short-term interactions do not generalize => degrade for long sequences
- Model cannot understand sequences longer than observed in training

Solution: Relative positional encodings

- Only/mostly relative distances matter => better generalization
- **BUT**: needs to be integrated into self-attention
- **Alternative: Rotary positional encodings (RoPE)**
  - Rotate token/word embeddings instead of adding positional encodings
  - See https://arxiv.org/abs/2104.09864 for further reading

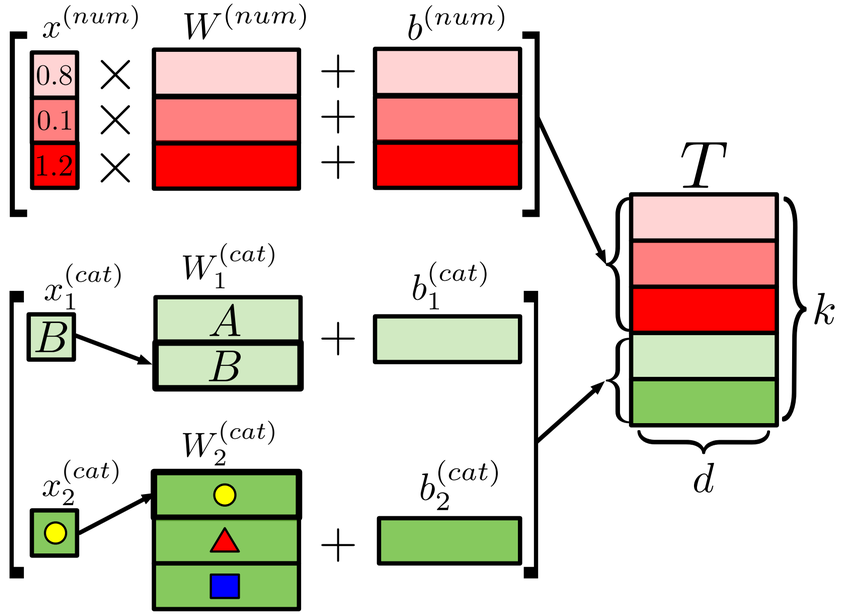
TUPLES / RECORDS AS SETS: COLUMN ENCODINGS

Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

Table records as sets

- Each of the columns $i \in [1, \ldots, k]$ is one element
  - Independent (non-shared) embedding layers per column
- Continuous columns $x_i \in \mathbb{R}$ (3 red examples)
  - Linear projection $W_i^{(\text{num})} \in \mathbb{R}^{1 \times d}$ to model dim d
  - Add bias $b_i^{(\text{num})} \in \mathbb{R}^d$
- Discrete columns $x_i \in \mathscr{C}_i$ (2 greend examples)
  - Embedding layer $W_i^{(\text{cat})} \in \mathbb{R}^{|\mathscr{C}_i| \times d}$ to model dim d
    (classes $\mathscr{C}_i$ and number of classes $|\mathscr{C}_i|$ may differ per column i
  - Add bias $b_i^{(\text{cat})} \in \mathbb{R}^d$



FT Transformer

Gorishniy et al. (2021)

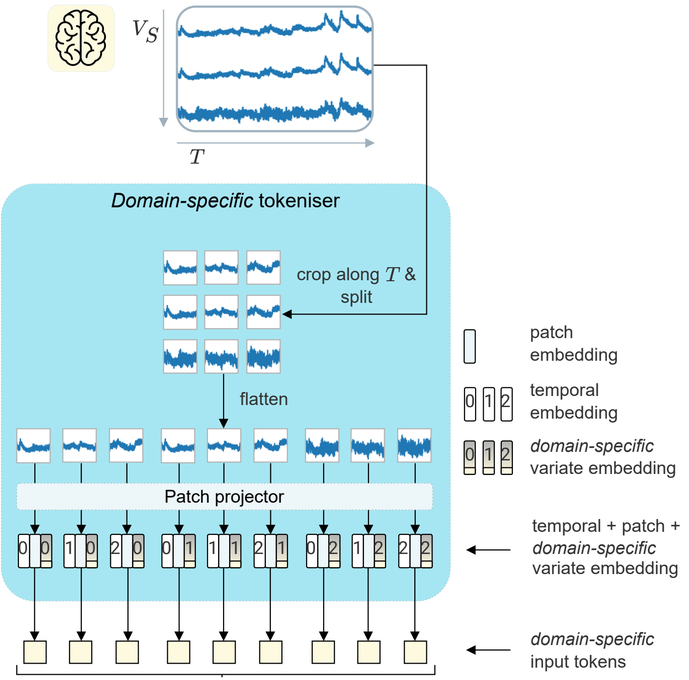
SIGNALS AS SETS: PATCHING AND CHANNEL ENCODINGS

Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

Signals / times series as sets

- Signal is split into patches of p time-steps
  - Some signals contain multiple (V) variates (channels)
  - 1 token per patch and variate = $T \times V$ tokens
- Patch embeddings
  - Patches are vectors of size p
  - Linear projection $W \in \mathbb{R}^{p \times d}$ to model dim d
- Add temporal encodings
  - Temporal encodings are positional encodings
- Add variate encodings
  - Variate encodings are learned per variate (channel)



OTiS

Turgut et al. (2024)

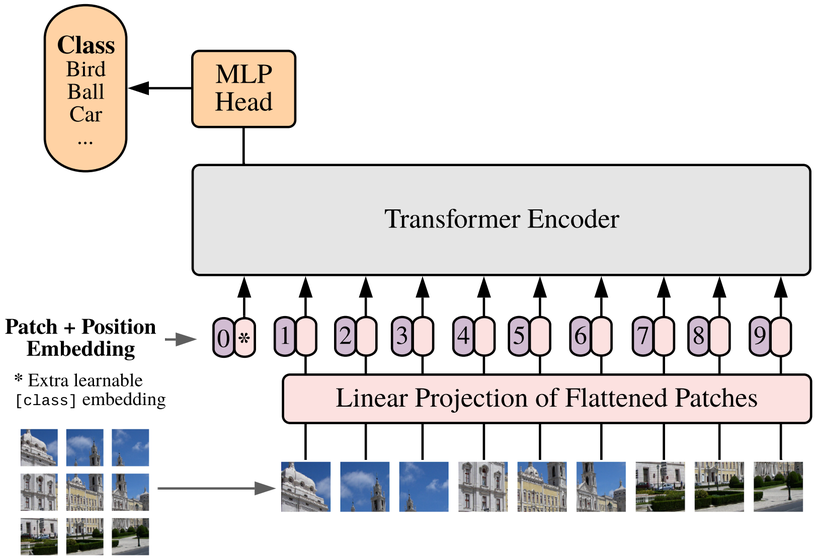
IMAGES AS SETS: PATCHING AND 2D POSITIONAL ENCODINGS

Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

Images as sets

- Image is split into patches of $p \times p$ pixels
  - Tokens = patches
- Patch embeddings
  - Flatten patch pixel intensities into vector of size $(p \cdot p)$
  - Linear projection $W \in \mathbb{R}^{(p \cdot p) \times d}$ to model dim d
- Add 2D positional encodings
  - Like normal (1D) positional encodings
    but capture structure of 2D Eucledian space



Vision Transformer (ViT)

Dosovitskiy et al. (2020)

ENCODING STRUCTURE AS FEATURES

Everything can be represented as a set!

➡ **Structure of data needs to be stored inside the features**

Even multimodal structures within multimodal models can be represented as sets!

Takeaways to encode any data type as a set

- Define elements (i.e. tokens), e.g. words, columns, words …
- Embed element features
- Encode structure, e.g. positions, 2D space, column types, …

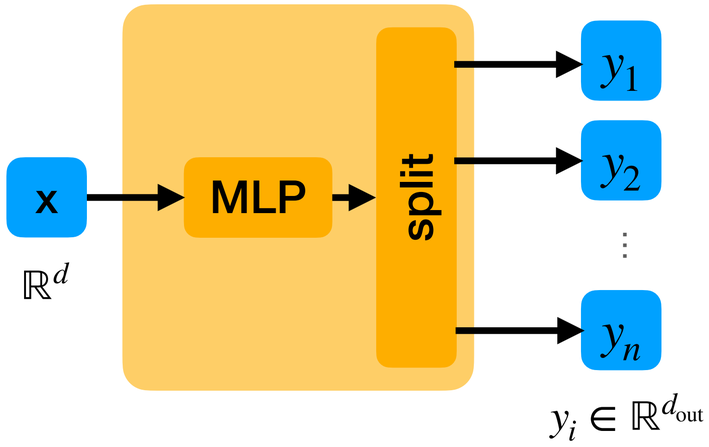
➡ **Select the appropriate elements, embeddings, structure encodings for your use case!**

Decoders

HOW TO GENERATE TUPLES

MLP + Split

1. Process your input and aggregate it to single vector ($\mathbb{R}^d$)
   - Use the appropriate encoder / aggregations layers
   - May be tuple/sequence/set input
2. Apply MLP on single vector
   - MLP : $\mathbb{R}^d \to \mathbb{R}^{n \cdot d_{\text{out}}}$
3. Split output dimension into individual vectors per element
   - Tuple length is fixed
4. (Optional) further process each tuple element
5. Apply loss function on each tuple element individually + average



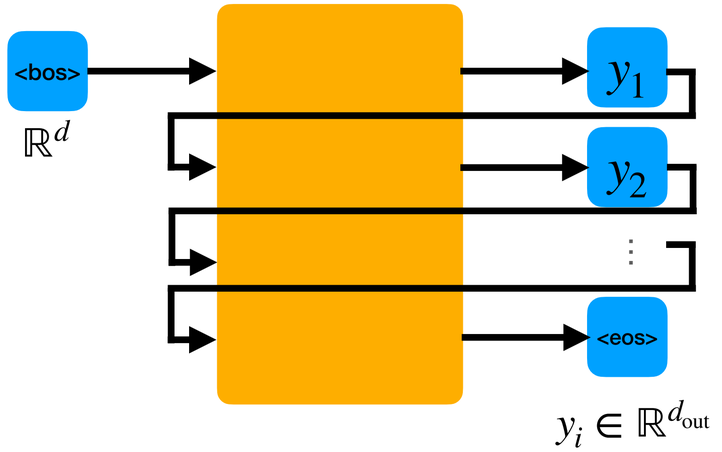
HOW TO GENERATE SEQUENCES

Problem: Dynamic output length

- How to convert fixed-size vector into dynamic number of vectors?

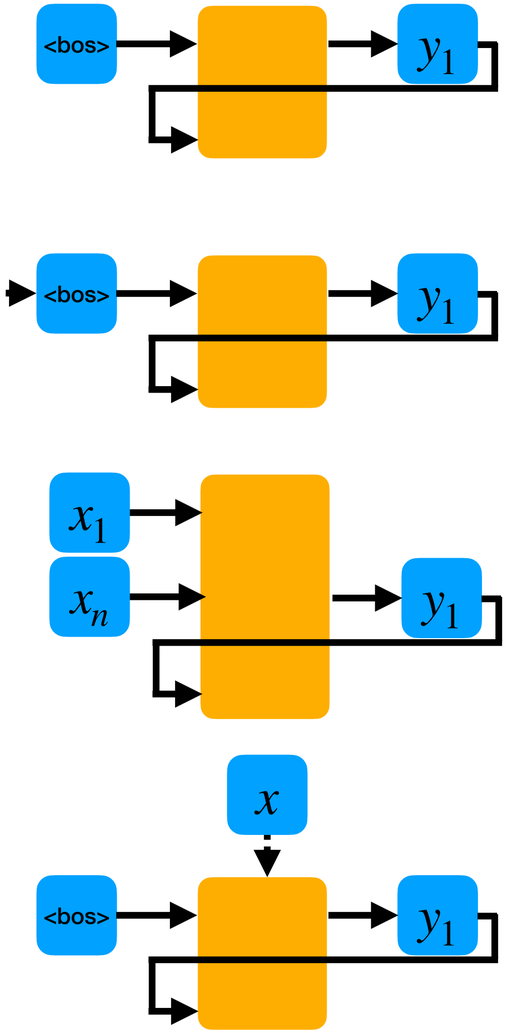
Solution: Generate step-by-step (Next Token Prediction)

1. Start from <bos> (begin of sequence) token
2. Predict next token conditioned on previous ones
   - **Causal model required!**
3. Stop generating when stopping criteria is reached
   - Stop when <eos> (end of sequence) token is predicted
   - Other criteria, e.g. max number of tokens



HOW TO GENERATE SEQUENCES — CONDITIONING ON ENCODER

- **No conditioning**
  - No encoder, simply generate sequences
  - Only use learned <bos> token
- **Single vector conditioning**
  - Information from encoder is aggregated into single vector
  - <bos> token conditioned on this vector
- **Prefix conditioning**
  - Multiple initial tokens (the prefix), conditioned on encoder
  - Prefix could contain one element for each input element (e.g. full input sequence / set is given to decoder, no aggregation)
- **Model conditioning**
  - Condition the model directly
  - Special decoder models required, e.g. with cross-attention
  - Support aggregated or full length encoder inputs



HOW TO GENERATE SEQUENCES — DECODER LAYERS

Self-attention-based decoder models

- Direct information flow from full context

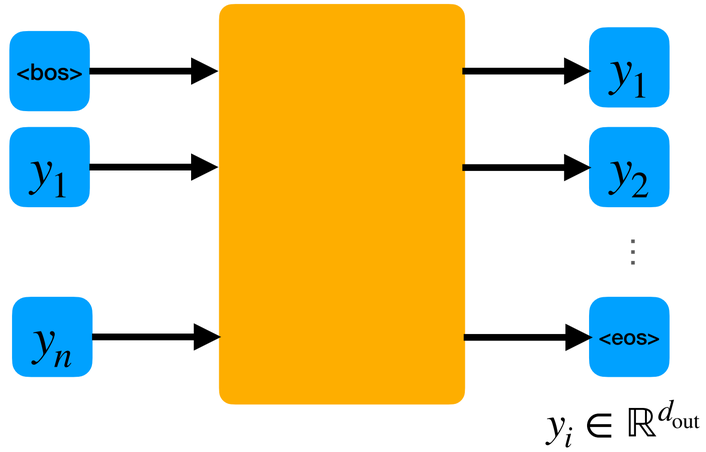
  ➡ works well with **single vector and prefix conditioning**

- Additional cross-attention layers required for **model conditioning**

HOW TO GENERATE SEQUENCES — TRAINING

Training with Teacher Forcing

- We know the full target sequence during training
  - Feed the target sequence shifted to the right
  - Prepand <bos>
- Apply the element-wise loss and average over sequence



How do we avoid looking into the future?

- We need a causal decoder model!

BUT: Self-attention is not causal by default!

HOW TO GENERATE SEQUENCES — TRAINING

Causal Self-attention

How to we make self-attention causal? **Attention masks!**

Attention masks

- Mask out the attention scores before applying softmax
  - Attention probabilities must be zero for pairs of query/key that where we dont want information to flow from value to query
  - It should appear as these pairs don’t exist -> no effect on other attention scores

=> Set these values to -inf => will lead to zero after exp

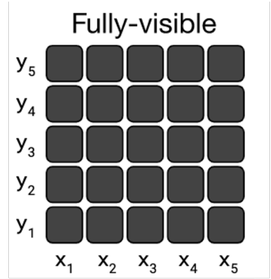
$$\mathbf{O} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\right)\mathbf{V}$$

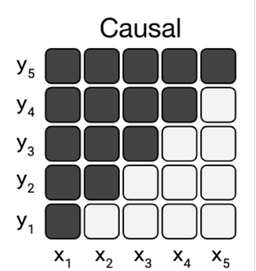
HOW TO GENERATE SEQUENCES — TRAINING

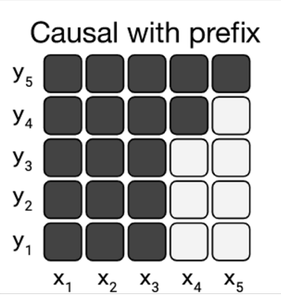
$$\mathbf{O} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\right)\mathbf{V}$$

Types of attention mask

- **Fully visible**
  - all outputs (queries) attend to all inputs (keys/values)
  - useful for encoders (same as no masking)
  - all-ones matrix
- **Causal**
  - outputs (queries) can only attend to previous inputs (keys/values)
  - useful for decoders (can't look into the future)
  - diagonal matrix
- **Causal with prefix**
  - fully visible for prefix inputs, causal mask for other inputs
  - useful for decoders with prefix conditioning
  - partly all-ones and partly diagonal matrix



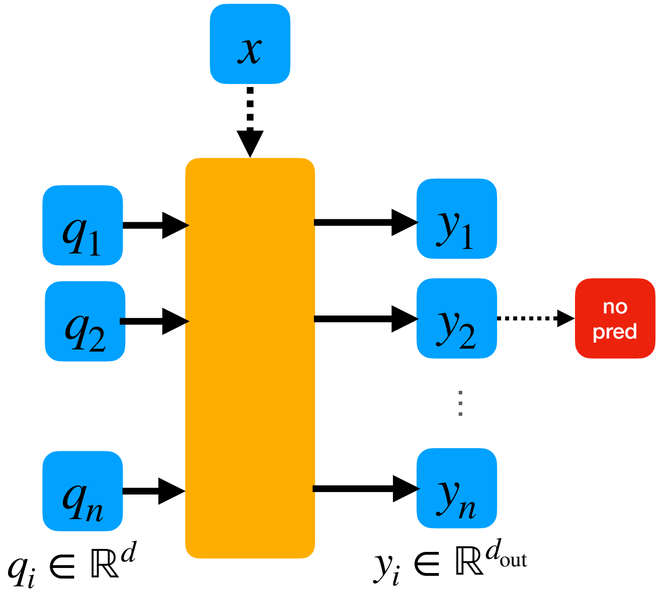




HOW TO GENERATE SETS

Problem: No ordering

- We cannot predict step-by-step without ordering
- Simple solution: Define canonical ordering



Solution without sorting: Predict all elements at the same time

1. Start from a set of query tokens
   - Learned query vectors
   - Number of query tokens = maximum size of predicted set
2. Apply set encoder on query tokens
   - Full (permutation-equivariant) interaction btw queries possible
   - Condition on encoder
3. Apply output prediction head on each query output individually
   - Prediction head can also predict “no-prediction” to enable dynamic sizes

HOW TO GENERATE SETS — TRAINING

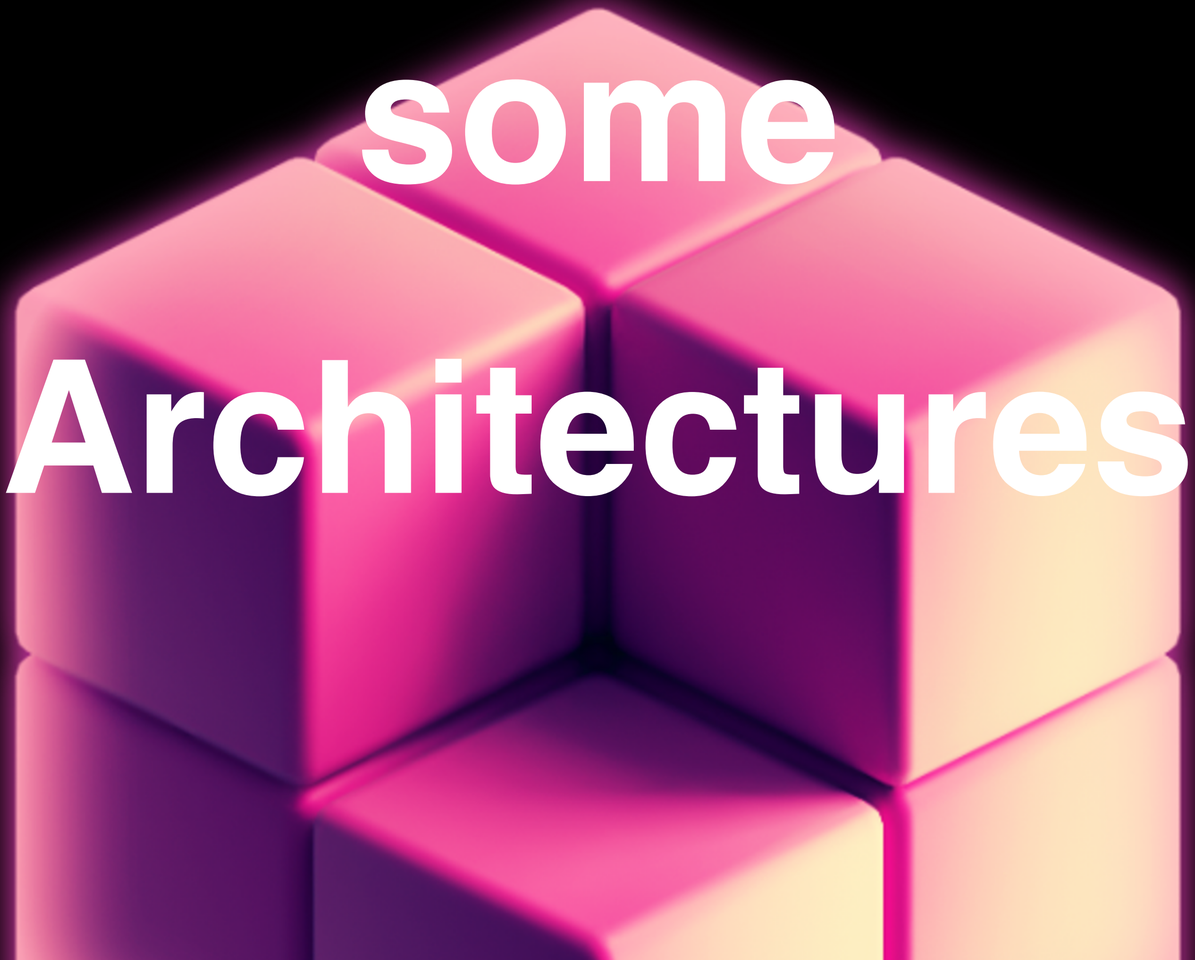
Problem: How to apply the loss function

- We know the target set
- **BUT: we don't know which predicted element belongs to which target element**
  - neither prediction nor target set has an order!

Solution: Matching

- Bipartite matching problem (assignment problem)!
- Define matching cost between predicted elements and target elements
  - based on similarity of prediction and target elements
  - e.g. by applying pairwise loss btw. each pair of prediction and target element
- Apply matching algorithm
  - typically **Hungarian matching**
- Compute final loss based on matched prediction-target pairs

Let’s look at some Architectures

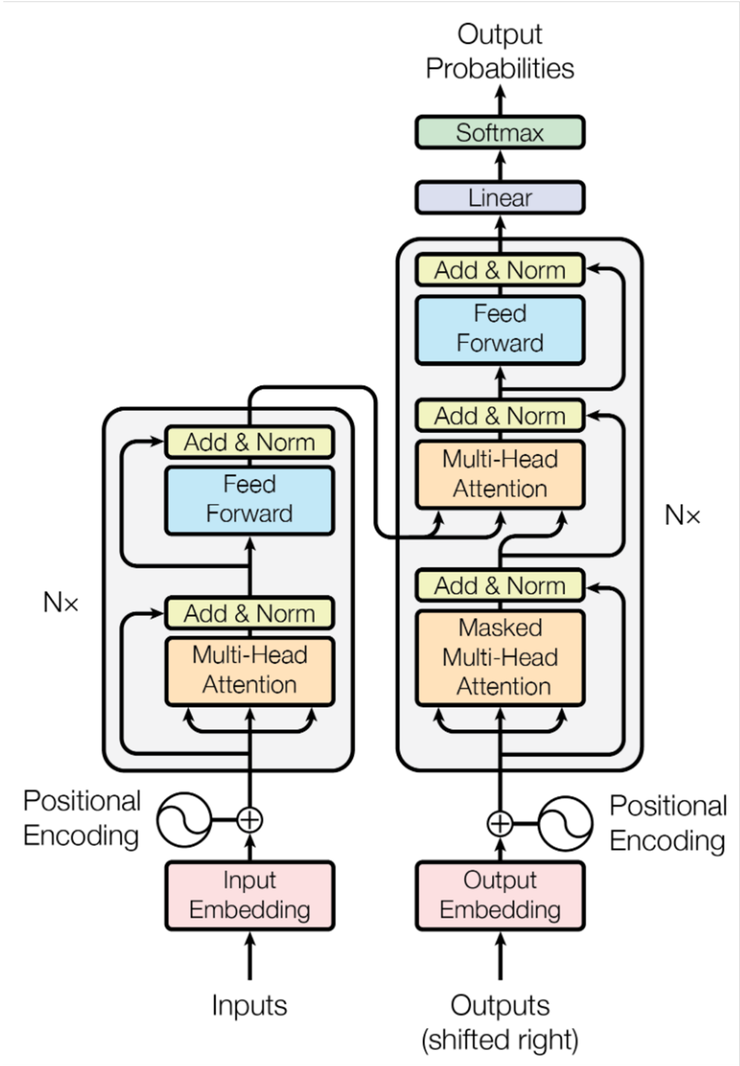


THE TRANSFORMER — A SEQUENCE-TO-SEQUENCE MODEL

Task: Translation (seq2seq)

- Input: Text (token sequence) in one language
- Output: Text (token sequence) in other language

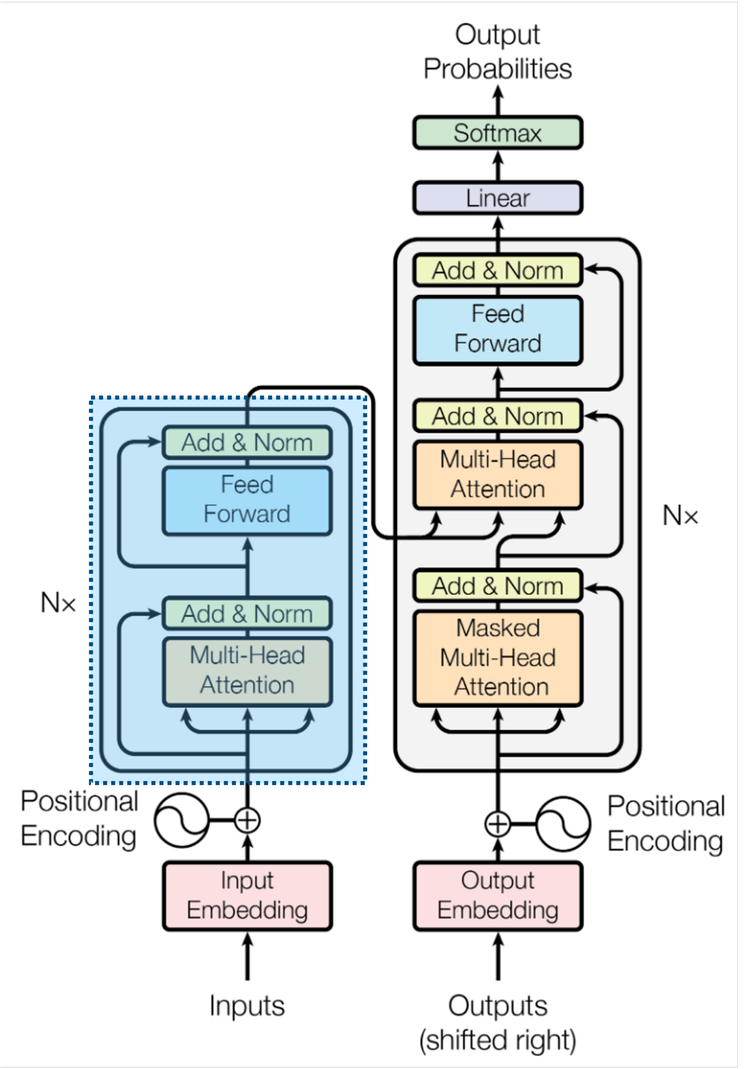
Architecture: Encoder-Decoder



THE TRANSFORMER — A SEQUENCE-TO-SEQUENCE MODEL

Encoder Blocks (Input Sequence)

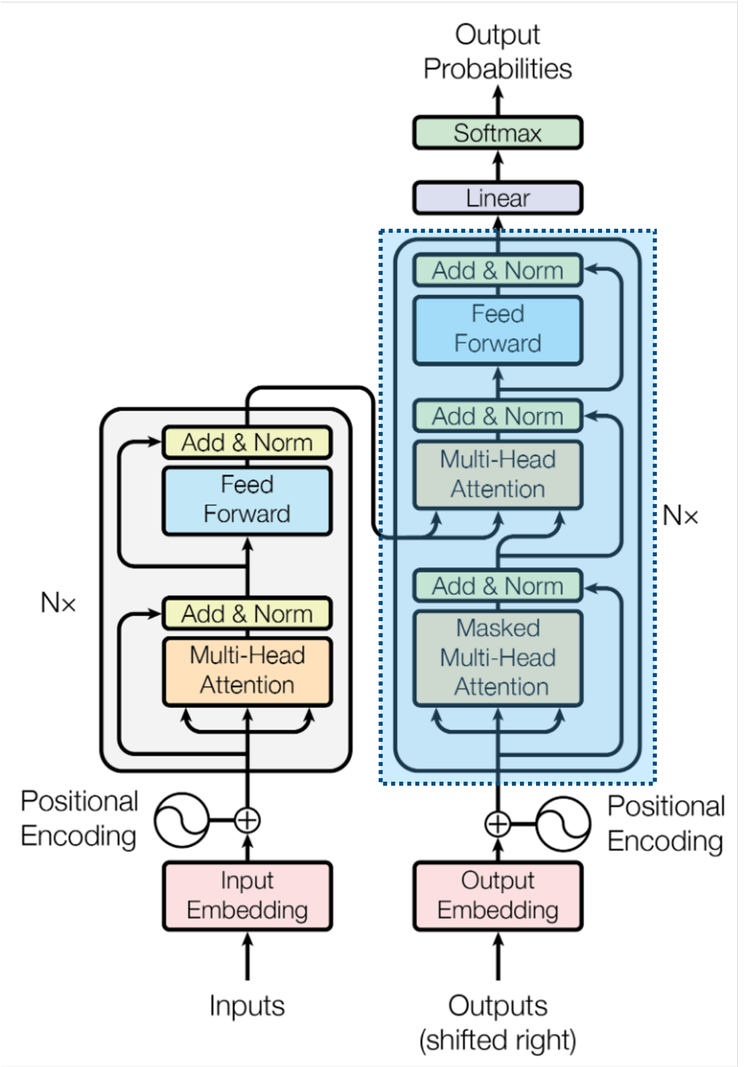
- **Multi-head Self-attention (MHA)**
  - Interaction between sequence elements
  - Full attention
  - Typically about 8 heads
- **Feed forward (FF)**
  - Element-wise operations (MLP)
  - Non-linear processing without interactions between sequence elements
  - Typically: 1 hidden layer with $d_{\text{ff}} = 4 \cdot d$
- **Residual (skip) connections and Layer Norm**
  - Applied around MHA and FF to training stability and deep networks
  - Modifications in newer versions (e.g., norm before, scaling, …)
  - **Very important components to make transformers work!**



THE TRANSFORMER — A SEQUENCE-TO-SEQUENCE MODEL

Decoder Blocks (Output Sequence)

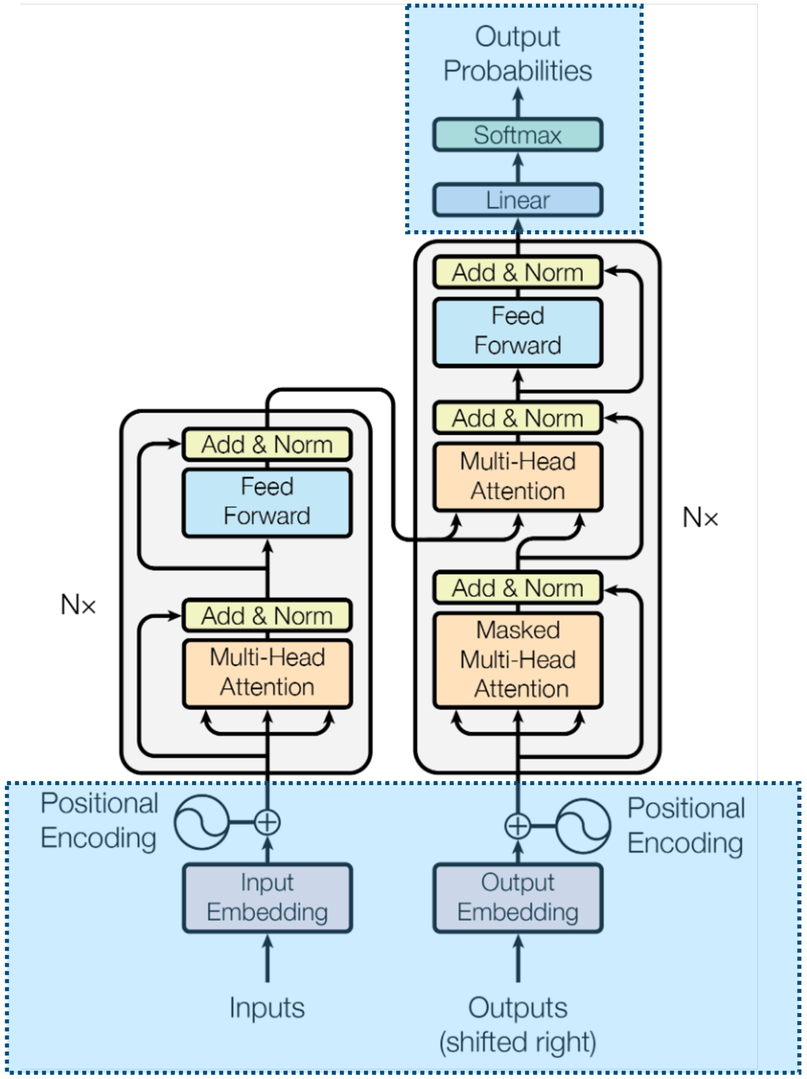
- **Multi-head Self-attention (MHA)**
  - Causal attention (avoid looking into the future)
  - Implemented by causal masking of attention scores
- **Multi-head Encoder-Decoder Attention (Cross attention)**
  - Queries from output sequence / MHA of same layer
  - Keys/Values from input sequence / same encoder layer
  - Conditioning output sequence on input sequence
- **FF, Residual Connections, Norms like Encoder**



THE TRANSFORMER — A SEQUENCE-TO-SEQUENCE MODEL

Overall Architecture

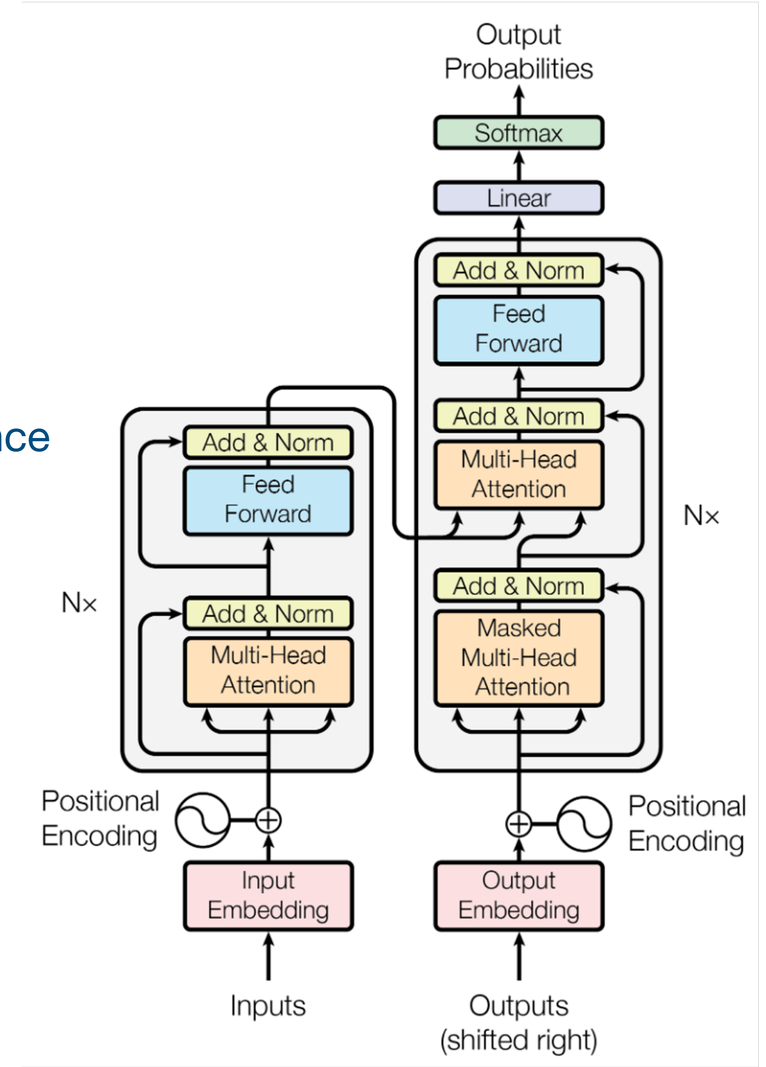
- **Token Embedding Layers (Input / Output)**
  - Learned embedding layer (token embeddings)
  - Convert token ID sequence (from tokenizer) to sequence of vectors
- **Positional Encoding**
  - Non-learned encodings of absolute sequence positions
- **Language Model Head (LM Head)**
  - Linear projection and softmax head
  - Predicts probability distribution over token IDs for each sequence element independently
  - Parameters may be shared with embedding layers
- **Encoder-Decoder Components**
  - Blocks can be stacked N times
  - cross-attention attention always within the same layer



THE TRANSFORMER — A SEQUENCE-TO-SEQUENCE MODEL

Modern Versions (LLMs)

- **Decoder only models!**
  - Drop the encoder model
  - Drop the cross attention
  - Keep causal masking
  - Tasks formulated as “autocomplete” => input and output as one sequence (input given, output predicted/completed)
- **Technical Optimizations**
  - Norms, rotary positional encodings, …
  - Sometimes: Attention restrictions and optimizations
- **Scaling, Training, Inference**
  - More layers, larger d, more heads
  - Parameter-efficiency, e.g. mixture-of-exports (MoE)
  - Autoregressive pre-training with additional reinforcement learning
  - Inference / sampling tricks

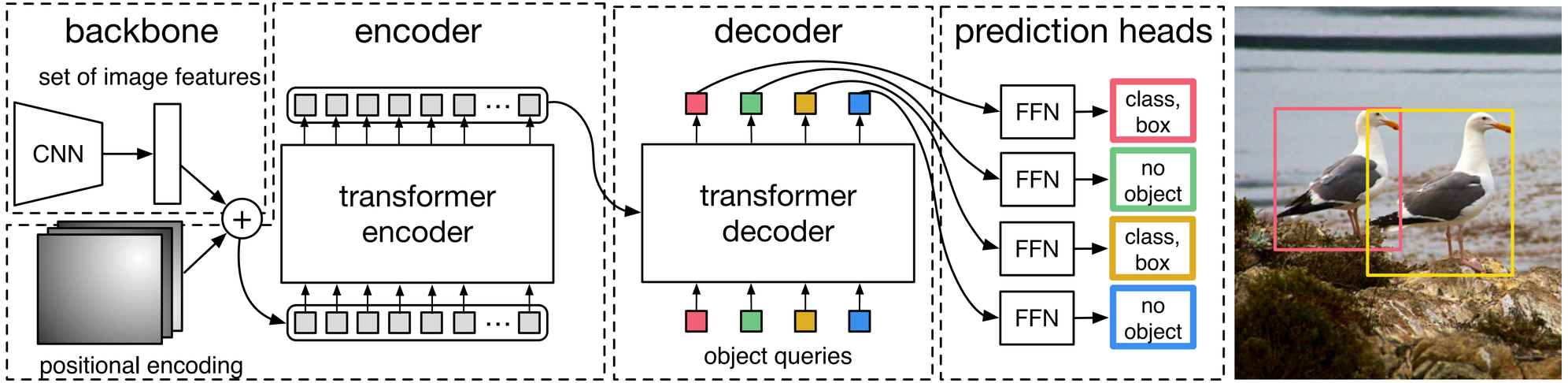


DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Task: Object Detection (set prediction)

- Input: Image
- Output: Set of bounding boxes (with classes)

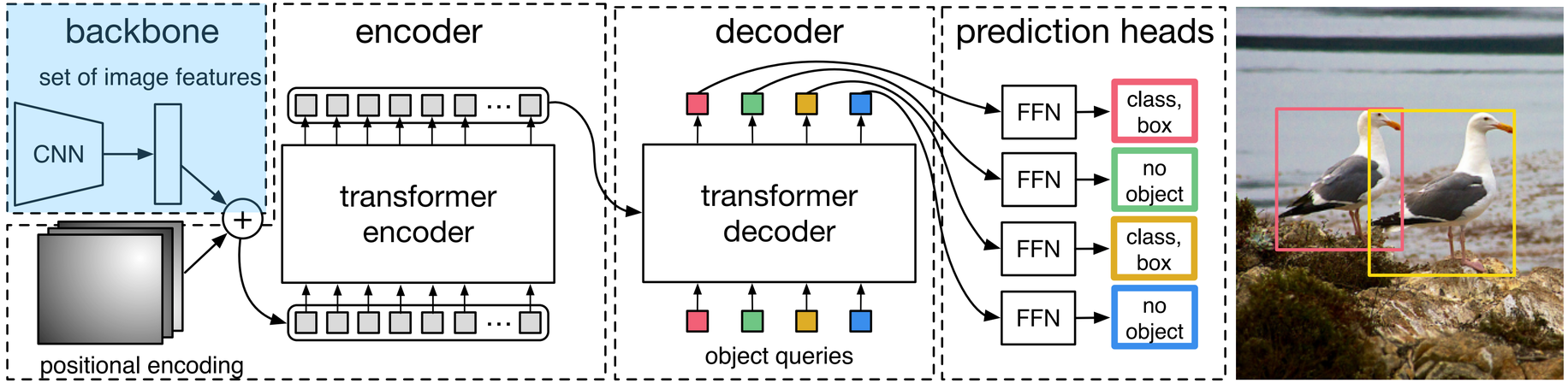
Architecture: Encoder-Decoder



DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Image Encoding (Backbone)

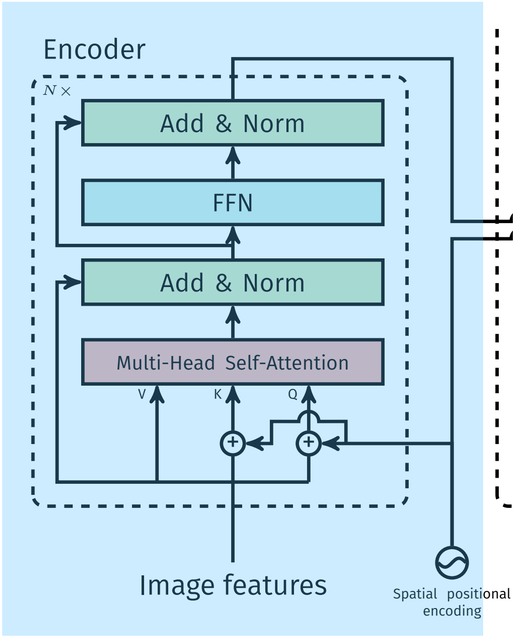
- Encode image tokens using CNN bakcbone + projection
- Add 2D positional embeddings

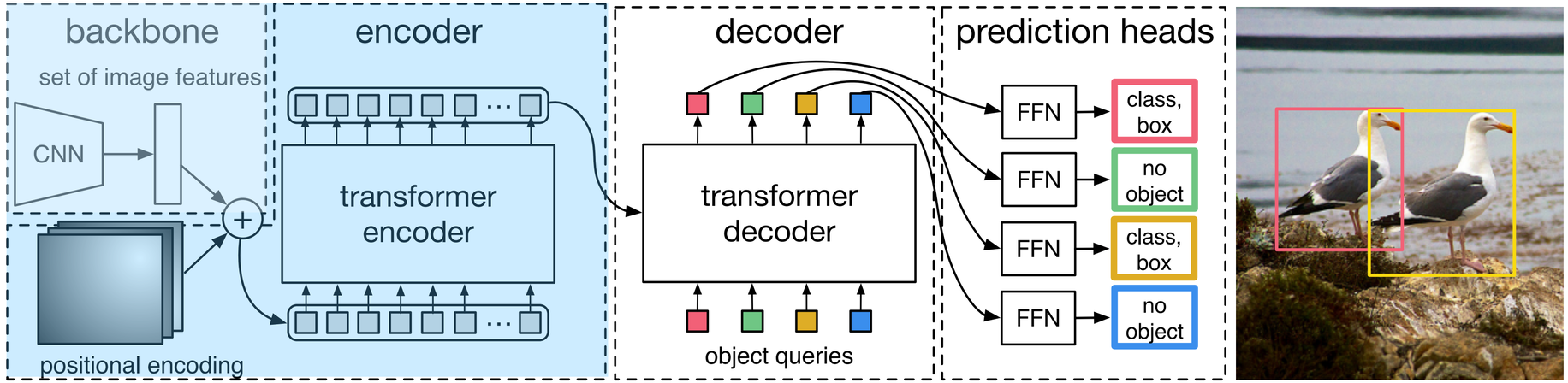


DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Encoder

- **Transformer encoder**
  - full self-attention
  - applied on image tokens
  - full interaction between all image tokens

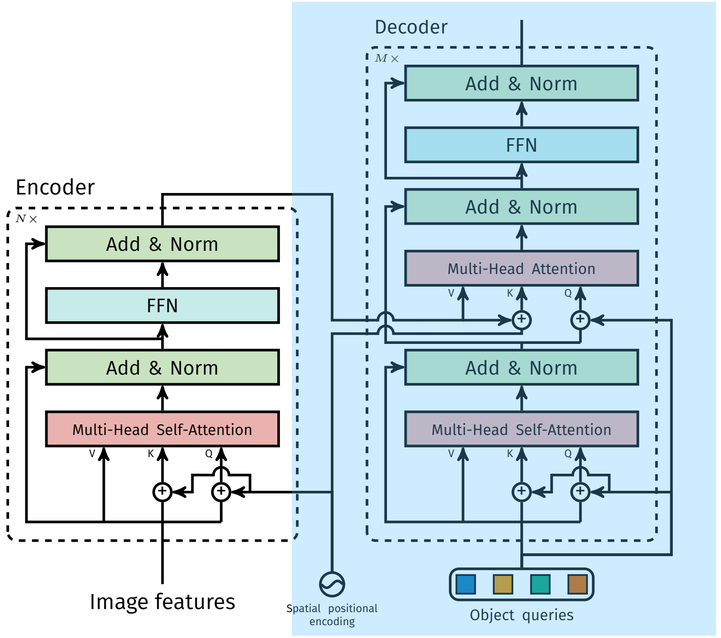


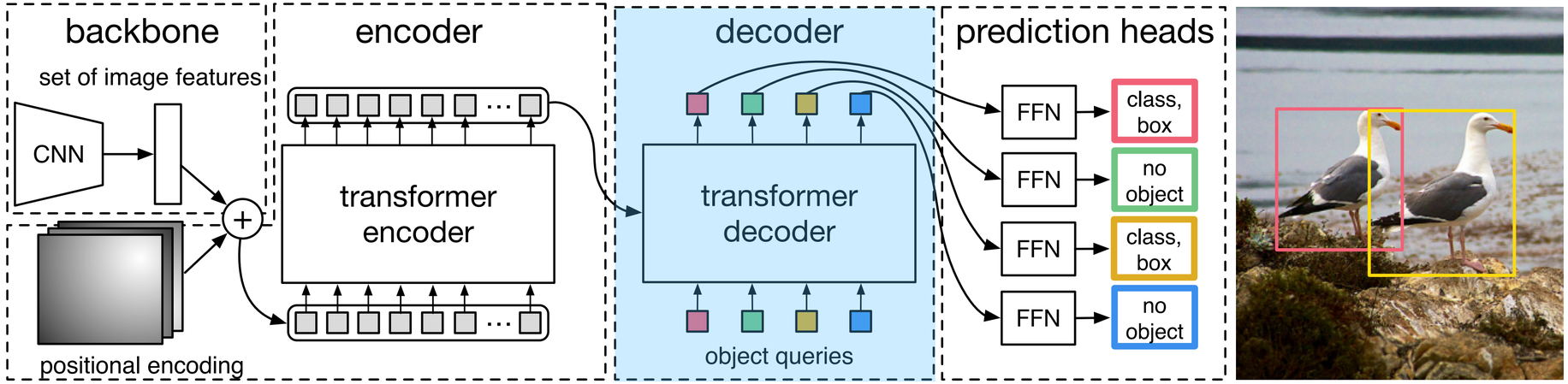


DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Decoder

- **Object queries**
  - Set of learned query vectors (number of queries = max. size of set)
  - 1 query = 1 predicted set element
- **Transformer decoder**
  - full self-attention and cross attention to encoder
  - queries = query tokens, keys/values = image tokens
  - full interaction between all query tokens and to all image tokens

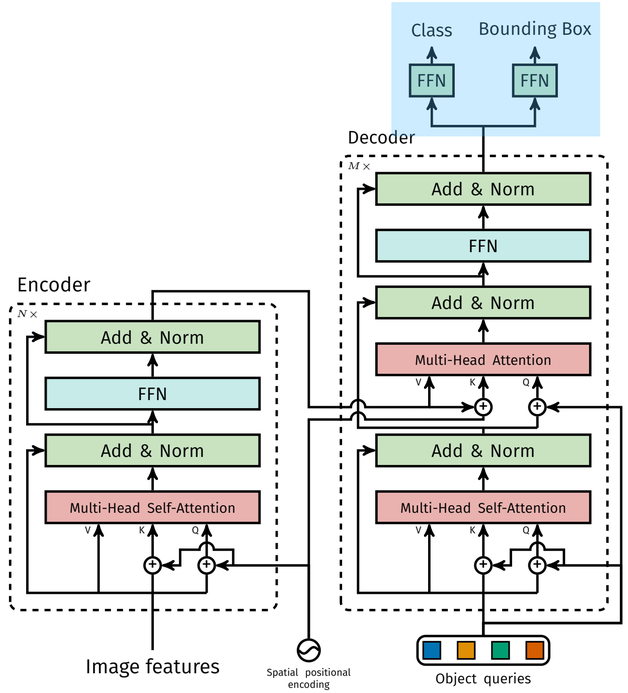


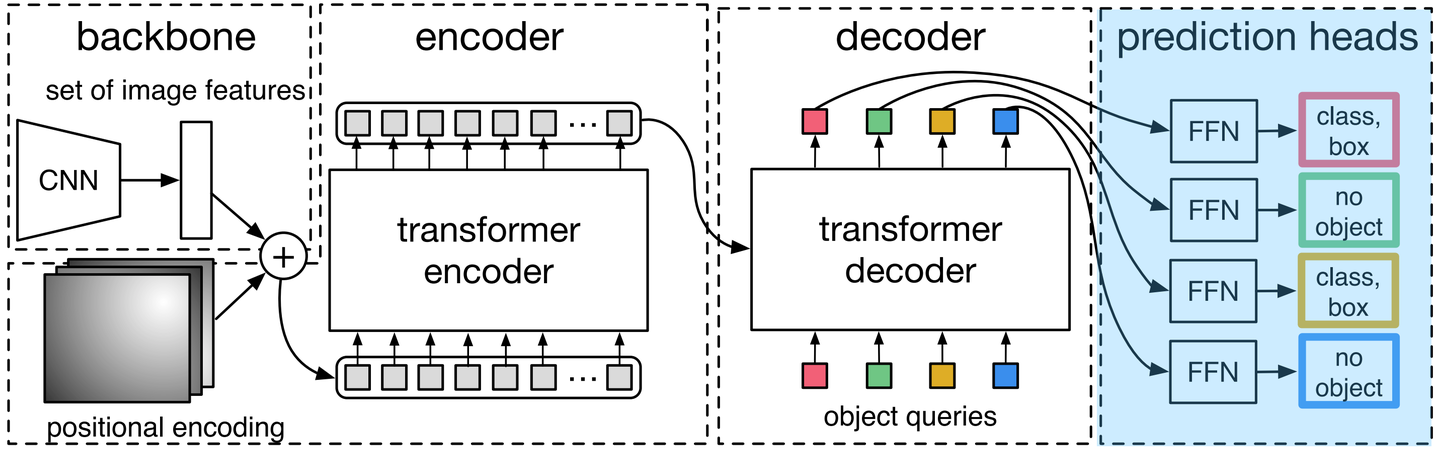


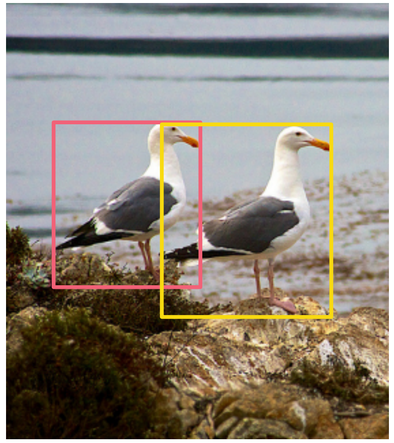
DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Prediction Heads

- **Class prediction**
  - Softmax-based class prediction per object query
  - Additional no-object class possible
- **Bounding box prediction**
  - Bounding box regressor
  - 4 coordinates (center and size)
  - Relative coordinates ([0, 1]) using sigmoid activations



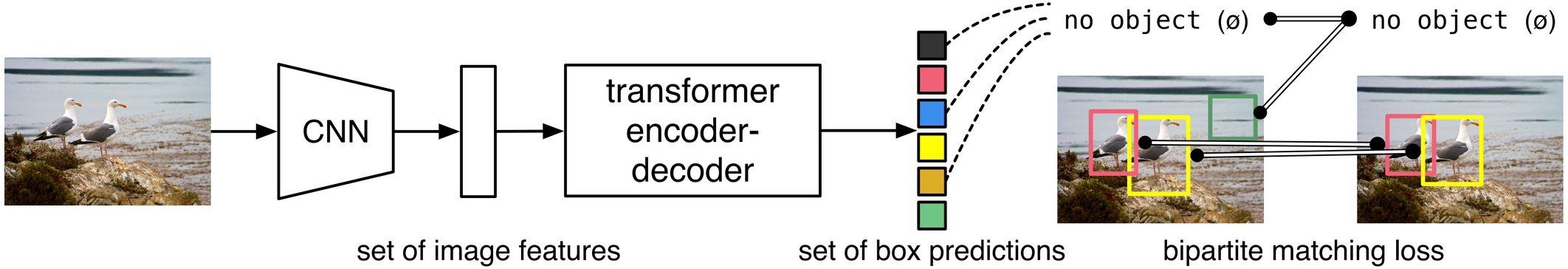




DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Training

- **Hungarian matching with targets**
  - Box overlap / similarity
  - Class similarity
- **Loss**
  - Applied on matched objects (box loss and class loss)
  - Push non-matched objects to predict no-object



DETR — TRANSFORMER-BASED SET DECODER FOR OBJECT DETECTION

Remaining Problem of DETR: Unstable Matching

- Hungarian matching is unstable in early training stages
- Slow convergence

➡ **Solution**: Priors for queries (e.g. queries based regions in the image)

➡ **Solution**: Other matching approaches

Tabular Data

WHY DOES DEEP LEARNING STRUGGLE WITH TABULAR DATA

On tabular data,
deep learning is often outperformed by
tree-based methods (e.g. XGBoost)!

BUT WHY?

WHY DOES DEEP LEARNING STRUGGLE WITH TABULAR DATA

Problem of Deep Learning on Tabular Data

- Tabular datasets are often small
  - Deep learning requires large amounts of data to perform weel (especially transformers)
  - Generalization between different tabular datasets is hard

    ➡ pre-training is not easy
- Properties of tabular datasets do not match inductive biases of deep learning
  - Deep learning is biased to overly smooth solutions
  - Deep learning tends to learn rotationally invariant features from tabular data (but some columns are independent, so this is undesirable)
- MLP-based approaches overfit on uninformative columns
  - Attention-based models work better here

DEEP LEARNING FOR TABULAR DATA

Further Resources (not relevant for this lecture)

- Why does deep learning struggle on tabular data?
  - https://arxiv.org/pdf/2207.08815
- Some architectures models:
  - TabFPN: https://arxiv.org/pdf/2207.01848
  - FT-Transformer: https://arxiv.org/pdf/2106.11959
  - TabNet: https://arxiv.org/pdf/1908.07442
- https://sebastianraschka.com/blog/2022/deep-learning-for-tabular-data.html

RECAP AND TAKEAWAYS

- **Key properties of Sets, Sequences, and Tuples**
  - Structure and permutation-equivariance
  - Properties of components
- **Encoder components**
  - Simple operations (element-wise, pooling, MLP, …)
  - Permutation-equivariant operations (Janossy pooling and attention)
  - Quadratic complexity of attention (and solutions)
- **Everything is a Set:** Structure encodings
- **Decoders:** Predicting sequences and sets
- **Architectures**
  - Transformer
  - DETR
- **Tabular Data**

RECAP AND TAKEAWAYS

- **Key properties of Sets, Sequences, and Tuples**
  - Structure and permutation-equivariance
  - Properties of components
- **Encoder components**
  - Simple operations (element-wise, pooling, MLP, …)
  - Permutation-equivariant operations (Janossy pooling and attention)
  - Quadratic complexity of attention (and solutions)
- **Everything is a Set:** Structure encodings
- **Decoders:** Predicting sequences and sets
- **Architectures**
  - Transformer
  - DETR
- **Tabular Data**

RECAP AND TAKEAWAYS

Identify the properties/structure of your data,
then select the encoder/decoder components accordingly
or encode the structure as features.

In practice we don’t always need to be too strict with equi-/invariances

- Some can be tackled with data augmentations
- Sometimes we want to use pre-trained models (e.g. LLMs)

BUT: Thinking about properties/structure is important

➡ it can improve (data) efficiency and generalization

RECOMMENDED READINGS

Recommended readings (not mandatory):

- Vaswani et al. **Attention Is All You Need** (2017)
  https://arxiv.org/abs/1706.03762
  (the original transformer paper)

- Carion et al. **End-to-End Object Detection with Transformers** (2020)
  https://arxiv.org/abs/2005.12872
  (the DETR paper)

FURTHER READINGS AND RESOURCES

Set Encoders, Janossy Pooling, and Attention:

- Fuchs et al. **Review: Deep Learning on Sets**
  https://fabianfuchsml.github.io/learningonsets/
- Wagstaff et al. **Universal Approximation of Functions on Sets** (2021)
  https://arxiv.org/abs/2107.01959
- Zhang: **Learning to Represent and Predict Sets with Deep Neural Networks** (2021)
  https://arxiv.org/abs/2103.04957
- Zaheer et al. **Deep Sets** (2017)
  https://arxiv.org/abs/1703.06114

FURTHER READINGS AND RESOURCES

Quadratic complexity of attention and solutions:

- Beltagy et al. **Longformer: The Long-Document Transformer** (2020)
  https://arxiv.org/pdf/2004.05150
- Zaheer et al. **Big Bird: Transformers for Longer Sequences** (2020)
  https://arxiv.org/abs/2007.14062
- Liu et al. **Lost in the Middle: How Language Models Use Long Contexts** (2024)
  https://direct.mit.edu/tacl/article/doi/10.1162/tacl_a_00638/119630
- Katharopoulos et al. **Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention** (2020)
  https://arxiv.org/pdf/2006.16236
- Tsai et al. **Transformer Dissection: A Unified Understanding of Transformer's Attention via the Lens of Kernel** (2019)
  https://arxiv.org/pdf/1908.11775
- Han et al. **Demystify Mamba in Vision: A Linear Attention Perspective** (2024)
  https://arxiv.org/abs/2405.16605
- Gu & Dao. **Mamba: Linear-Time Sequence Modeling with Selective State Spaces** (2023)
  https://arxiv.org/abs/2312.00752
- Behrouz et al. **Titans: Learning to Memorize at Test Time** (2024)
  https://arxiv.org/pdf/2501.00663
- Dao et al. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness** (2022)
  https://arxiv.org/abs/2205.14135

FURTHER READINGS AND RESOURCES

Structure as Features:

- Su et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding** (2021)
  https://arxiv.org/abs/2104.09864
- Gorishniy et al. **Revisiting Deep Learning Models for Tabular Data** (2021)
  https://arxiv.org/pdf/2106.11959v5
- Turgut et al. **Towards Generalisable Time Series Understanding Across Domains** (2024)
  https://arxiv.org/abs/2410.07299
- Dosovitskiy et al. **An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale** (2020)
  https://arxiv.org/abs/2010.11929

Tabular Data:

- Grinsztajn et al. **Why do tree-based models still outperform deep learning on tabular data?** (2022)
  https://arxiv.org/pdf/2207.08815
- Hollmann et al. T**abFPN: A Transformer that solves small tabular classification problems in a second** (2022)
  https://arxiv.org/pdf/2207.01848
- Arık & Pfister. **TabNet: Attentive Interpretable Tabular Learning** (2019)
  https://arxiv.org/pdf/1908.07442